

Implementation Plan — Modular friction/nucleation/Riemann for the SAFS spatial driver, with byte-exact TPV205/104/102/31 regression and a rate-and-state SAFS option

SEAS-MFEM · fullelasticity_dev / parts_dev

2026-05-25 (rev 6 — sixth pass; addresses REVIEW.md round-2 R-009...R-011)

Contents

1	§0 Review reconciliation (rev 6 — REVIEW.md R-001...R-011)	4
2	Overview	6
3	Resolved decisions (was: open questions)	6
3.1	D1 — Fault-basis source → KEEP safes's; add a planar-TPV correctness test	6
3.2	D2 — CFL safety → adopt opt-in cfl_safety, keep BOTH options	7
3.3	D3 — Stress-sign convention → two complementary moves	7
3.4	D-rename — general method name → DO the rename + consistency check	9
4	Byte-exactness contract (the non-negotiable)	10
5	Git provenance, reference, and existing scaffolding	11
5.1	RS scaffolding already in safs (the SAFS+RS head start)	12
6	Module architecture (the “parts”)	13
6.1	friction_laws part (unified, method-oriented)	13
6.2	nucleation_methods part	15
6.3	riemann_flux part (homogeneous scalar vs heterogeneous matrix)	16
6.4	Shared-rank correctness constraints (must hold for every fault-flux / exchange change)	17
6.5	What leaves vs stays in the driver	17
7	Code graph — full structure	18
7.1	G1 — Code map (files by role)	18
7.2	G2 — Runtime wiring (setup: config -> objects)	19
7.3	G3 — Per-macro-step call & data flow (time loop)	19
7.4	G4 — Stress-source dispatch (both paths: positive input = right-lateral on-fault) . .	20
7.5	G5 — Riemann-flux dispatch (scalar vs matrix; fault flux unaffected)	20

8	Master phase / file table	21
9	Phases	23
9.1	Phase 1 — RS primitives » (aging-iterator hooks + equilibrium- ψ seed + build) . .	23
9.2	Phase 2 — Friction strategy interface » (IFrictionIterator + adapters + factory) .	27
9.3	Phase 3 — SAFS + rate-and-state run » (driver RS branch + D3.1 + config) — THE NEAR-FUTURE GOAL	30
9.4	Phase 4 — Name generalization (D-rename)	35
9.5	Phase 5 — Unified method-oriented iterator (replaces the Phase-2 adapters)	35
9.6	Phase 6 — TPV/material/flux config schema + D3.2 fault-local stress	36
9.7	Phase 7 — nucleation_methods module	38
9.8	Phase 8 — TPV205/102/104 benchmarks (configs, meshes, jobs, regression)	39
9.9	Phase 9 — Heterogeneous (matrix) Riemann solver option (PORT from hrs-ref) . .	39
9.10	Phase 10 — TPV31 (depth-heterogeneous LSW) verification case (PORT from hrs-ref)	41
10	Testing strategy	42
11	Risk assessment	43
12	Appendix	45

STATUS: DRAFT — APPROVAL REQUIRED. NO IMPLEMENTATION until this pass is signed off. This is rev 6 of the TPV-via-spatial_dyn_driver plan. It keeps the SAFS structure, wires the TPV benchmarks, and adds the missing functionalities as clean reusable modules.

Rev-6 deltas (this pass — the round-2 re-audit REVIEW.md R-009/R-010/R-011): - **R-009 (MODERATE→CRITICAL if $v_0 \neq 1e-6$):** the force solve hardcodes Friction-Solver::V0 = 1e-6 while the RS seed + aging law use config blk.V_0_default — a silent dual-source. Phase 1 SeedEquilibriumPsi_RS now asserts they agree (defensive copy in the factory); the deliverable keeps $V_0 = 1e-6$. Proper fix (thread config V0 into the solver, per the no-hardcoded-numbers rule) is a documented follow-up gated out of the byte-exact TPV oracle path. - **R-010 (MODERATE — would not compile):** Phase-2 req 5 wrongly said the body “already calls iterator.Advance(…).” It calls Advance-WithSubStepStates(…) at driver :439, and also GetDeltaT()/GetTimeWeights() at :384/:385/:404. Rev 6 renames the :439 call to Advance(…) **and** adds the two getters to IFrictionIterator (the reviewer’s narrower fix would still not compile). - **R-011 (LOW/POSSIBLE):** ResolveRateState:1070 hard-aborts unless $a < b$, forbidding the velocity-strengthening border a bounded tpv102-like rupture needs. Rev 6 relaxes it to allow $a > b$ (keep $a, b > 0$). Optional for the fully-VW first smoke; required before a bounded run. - **Cleared (no change):** Makefile inline-link sufficiency, AgingLawPsi ctor order, driver RS-resolver scoping — all verified correct by the reviewer (recorded in §0).

Rev-5 deltas (carried — addresses the plan-level audit in REVIEW.md round 1): every finding below is now closed in the priority phases (full mapping in §0 Review reconciliation): - **R-001 (CRITICAL):** SeedEquilibriumPsi_RS pinned to d.eta_s (=

rs.eta(i) = 0.5*sqrt($\mu\rho$) for homogeneous SAFS) — **never** rs.eta_s (no such member); added the missing #include `"../friction/dieterich_ruina.hpp"` and the in-body `V_init>0` guard. - **R-002 (CRITICAL)**: Phase-3 resolver branch now **gates** the ungated MFEM_VERIFY(`cfg.slip_weakening.has_value()`) at driver :1146 under `is_lsw`, and declares both `lsw` and `rs` at **outer scope** so the DOF-init at :1220 still sees them. - **R-003 (MODERATE)**: ResolveRateState is now called with a **zero** PorePressureSpec{} (`geom.sigma_n_per_dof()` is already effective $\sigma_n - P_p$) — no double pore-pressure subtraction. - **R-004 (MODERATE)**: the shared-fault RS ψ accuracy gate is reframed as a **temporal** (interior-vs-shared ψ) check plus the existing cross-rank bit-identity check; `np=2` test added. - **R-005 (MODERATE)**: Phase-3 branches the ParaView “state” channel to write `d.psi` for RS (the LSW `LSWFrictionCoefficient_TPV205` path is 0/NaN for RS). - **R-006 (MODERATE)**: ResolveRateState now **rejects** per-DOF `b/V_0/f_0` spatial overrides (the aging law + `DOFData` are scalar in those) — making the plan’s claim true. - **R-007 (LOW)**: D3.1 **kept in Phase 3** per the user’s standing decision, but hardened — factory negation + **all** config flips (`Dc2/Dc8/Dc10` + RS) land in **one atomic commit**, and the golden sign test is **parametrized over every flipped config**. - **R-008 (LOW)**: the `tpv102` parity AC now **pins NewtonRaphsonStable on both overloads** (production uses Brent); a separate Brent test checks finiteness/monotonicity only.

Rev-4 deltas (carried — the prioritization that rev 5 hardens): 1. **SAFS + rate-and-state is now the near-future goal, delivered in Phases 1–3** ». A source audit (§4.1) shows the **entire RS config/resolver/setup/kernel/iterator/wave-op dispatch already exists** in `safs` — SAFS+RS is a *driver-wiring* job. Phases 1–3 close the small gap (a `Tpv102` nucleation-callback overload, an equilibrium- ψ seed, an `IFrictionIterator` strategy with adapters over the **existing proven** iterators, and the driver RS branch); everything else (unification, TPV regression, matrix Riemann, TPV31) follows in Phases 4–10. 2. **D3.1 unchanged this pass** (the user confirmed “stick with current plan”): the `constant_tensor` input is right-lateral-positive via a factory-level negation that leaves the stored tensor and all on-fault quantities bit-identical; it now lands in **Phase 3**. 3. **Expanded code + unit-test detail** for the prioritized phases — concrete signatures (`SeedEquilibriumPsi_RS`, the `Tpv102` overload, `IFrictionIterator/adapters/factory`, the RS driver branch) and bit-for-bit/residual test specs.

Carried from rev 3 (still in scope, later phases): 1. **Friction iterators unified, method-oriented (§5.1)**. The three benchmark-named sub-step iterators collapse into one templated `RateStateSubStepIterator<StatePolicy>` (aging / slip-law-SRW differ by *one* policy line) plus a separate `LinearSlipWeakeningIterator`, sharing one `RunSubSteps_` skeleton — byte-exact to the standalone drivers, ~250 duplicated lines removed. 2. **D3 rewritten (§2-D3)**. The fault-local TPV option is kept; *additionally* the regional `constant_tensor` projection now takes **right-lateral-POSITIVE** shear input and yields right-lateral-positive on-fault shear, with a full global $\leftrightarrow$ local derivation and a **bit-identity** proof (the stored Cauchy tensor and all on-fault quantities are unchanged; the no-flip projection rule R-501 is *not* reverted). 3. **Heterogeneous (matrix) Riemann solver enabled as a first-class option (§5.3, Phase 9)** alongside the homogeneous (scalar) solver; mixed-flux remains scalar-only. **TPV31** (depth-heterogeneous material) is added as the matrix-path verification case (Phase 10). 4. **Proposed layout simplified**

(§5, §6) — design only; no real-code edits in this pass. 5. **Shared-rank correctness constraints made explicit (§5.4)** and threaded through every phase that touches a fault flux or a cross-rank exchange.

1 §0 Review reconciliation (rev 6 – REVIEW.md R-001...R-011)

Every finding from the plan-level audit (REVIEW.md, rounds 1–2, 2026-05-25) was checked against live source (greps in the notes below) and is closed in the phase shown. Severity and “blocks the deliverable?” are the reviewer’s. **Round 2 added R-009/R-010/R-011 and cleared three round-1 suspicions** (see the end of this section); R-001...R-008 carry over from rev 5.

ID	Sev	Where	Resolution in this rev	Phase	Test
R-001	CRIT	SeedEquilibriumPsi	Use <code>rs.eta_s (= rs.eta(i)); not rs.eta_s (absent)</code> . Add <code>#include dieterich_ruina.hpp</code> ; add in-body <code>V_init>0 abort</code> .	1	seed_equilibrium_psi_r (extended)
R-002	CRIT	driver :1146	Gate <code>slip_weakening.has_value()</code> under <code>is_lsw</code> ; declare <code>lsw+rs</code> at outer scope; RS branch asserts <code>rate_state.has_value()</code> .	3	rate_state_config_no_a (dry-run)
R-003	MOD	ResolveRateState	Pass <code>PorePressureSpec{}</code> (input already effective $\sigma_n - P_p$); documents the <code>sigma_n_total_per_dof</code> trap.	3	resolve_rate_state_no
R-004	MOD	shared-fault ψ	Reframe gate as temporal (interior-vs-shared ψ) + cross-rank bit-identity; bounded (mirrors LSW slip path); <code>np>1</code> RS stays “open until validated”.	3	shared_fault_rs_psi_co
R-005	MOD	ParaView state writer :1809	Branch: RS writes <code>d.psi</code> ; LSW path byte-unchanged.	3	paraview_state_channe
R-006	MOD	resolve_rs_i	Reject <code>per-DOF/10180/f_0</code> (aging law + <code>DOFData</code> scalar) — makes the plan’s claim true.	3	resolve_rate_state_re
R-007	LOW	D3.1 multi-config flip	Keep D3.1 in Phase 3 (user’s standing decision) but land factory negation + all flips (<code>Dc2/8/10 + RS</code>) atomically ; golden test parametrized over every flipped config.	3	constant_tensor_sign (parametrized)

ID	Sev	Where	Resolution in this rev	Phase	Test
R-008	LOW	tpv102 parity AC	Pin NewtonRaphsonStable on both overloads for EXPECT_EQ; separate Brent finiteness test.	1	tpv102_nuc_callback_pa (clari- fied)
R-009	MOD→CRF if V0≠1e- 6	FrictionSolver vs config V_0	Add: V_0 == FrictionSolver::V0 guard in SeedEquilibriumPsi_RS (+ defensive in factory); deliverable keeps V_0=1e-6. Proper fix (thread config V0 into the solver) is a gated follow-up.	1	seed_equilibrium_psi_r (V0 case)
R-010	MOD (build)	Phase-2 retype vs driver :439	Rename the :439 AdvanceWithSubStepStates(...) call → Advance(...); and add GetDeltaT/GetTimeWeights to IFrictionIterator (body calls them at :384/:385/:404).	2	advance_interface_comp
R-011	LOW (POSSI- BLE)	ResolveRateState :1070	Relax :1070 b → allow a > b (velocity-strengthening; keep a, b>0). Optional for the fully-VW first smoke; needed for a bounded tpv102-like rupture.	3	resolve_rate_state_al

Source-grounding notes (verified 2026-05-25): RateStatePerDOFParams member list is Vector a,b,Dc,V_init,f_0,V_0,eta,sigma_n_eff (spatial_friction.hpp:295) — **eta, no eta_s**; DOFData.eta_s is set at spatial_setup.hpp:82,128. Un-gated guard confirmed at spatial_dyn_driver.cpp:1146; ResolveSlipWeakening at :1151; InitializeFaultDOFs_Spatial at :1220. PorePressureSpec is a struct (spatial_friction.hpp:72); ResolveRateState takes const PorePressureSpec& pp (:313/:325). ParaView LSW writer at spatial_dyn_driver.cpp:1809. resolve_rs_impl applies per-DOF b/f_0/V_0 with **no rejection** at spatial_friction.cpp:1014/1017/1018. tpv102 plain overload defaults method = NewtonRaphsonStable (tpv102_substep_iterator.hpp:112,127). **Round 2:** FrictionSolver::V0 = 1e-6 is static constexpr (friction_solver.hpp:60), consumed by the force solve (fault_face_flux.cpp:224); AdvanceADERWithSubStep_Spatial's body calls iterator.GetDeltaT()/GetTimeWeights()/SetSubSteps()/AdvanceWithSubStepStates() at :384/:385/:404/:439 (so the interface needs the two getters **and** the :439 rename); the a < b validator is spatial_friction.cpp:1070. Both Tpv205/Tpv102 iterators expose GetDeltaT/GetTimeWeights (tpv205...hpp:138-139, tpv102...hpp:130-133).

The reviewer's R-001 quoted rs.eta_s; the live .md already used d.eta_s — rev 5 keeps d.eta_s (the reviewer's “more robust” choice, exactly what FaultFaceFlux consumes) and adds an explicit “**never rs.eta_s**” guard-comment so a fix agent cannot drift to the bad member. **On R-010:** the reviewer flagged only the :439 rename, assuming GetDeltaT/GetTimeWeights were already on the interface; they were **not** in the rev-5 draft, so rev 6 adds them too — a deliberate over-fix to make the retype actually compile.

Round-1 suspicions the reviewer cleared (no change needed — recorded so they are not re-litigated): (a) linking only `TPV102_SUBSTEP_ITERATOR_OBJ` is sufficient (`ApplyNucleationIncremental_TPV102` is inline); (b) `AgingLawPsi(b, v0, f0)` ctor order matches the Phase-2 adapter — no arg swap; (c) `dof_to_elem/dof_to_attr/dof_coords_3d/ material/pmesh` are all populated and in scope before the RS resolver call.

2 Overview

Today the SAFS dynamic-rupture driver `miniapps/seas/drivers/spatial_dyn_driver.cpp` runs **only** linear slip weakening (LSW) and **rejects** `[meta].law = "rate_state"`. This plan keeps that driver and its SAFS-specific fixes as the backbone and:

1. **Factors friction laws, nucleation methods, and Riemann solvers into reusable modules** — `friction_laws = {LSW, rate-and-state aging, rate-and-state slip-law strong rate-weakening (SRW)}`, `nucleation_methods = {static/spontaneous overstress, gradual overstress (Gaussian), gradual overstress compact-circular, instantaneous circular overstress}`, `riemann_flux = {homogeneous scalar Godunov, heterogeneous matrix (bimaterial)}` — sharing the existing ADER, `FaultBasis`, impedance, and per-DOF resolver infrastructure.
2. **Wires four SCEC benchmarks** `TPV205` (LSW), `TPV102` (RS aging), `TPV104` (RS SRW), and `TPV31` (LSW on a depth-heterogeneous medium → matrix Riemann) through that driver via per-case TOMLs, with physics **byte-exact** against the verified standalone drivers `drivers/tpv{205,104,102}_driver.cpp` (and, for `TPV31`, the `hrs-ref` implementation).
3. **Adds a rate-and-state SAFS run:** the existing `jobs/safs/spatial_dyn_resolution_Dc2_8N_400r_dev_2hr_s` problem, but with user-defined RS parameters from a TOML (aging-law, “`tpv102-type`”).

Central facts (verified). (a) The three primary TPVs (205/102/104) are homogeneous, so they run at `[numerics] interior_flux = "scalar"` — the existing `safs` scalar `WaveOperator` path, untouched. (b) **TPV31 is depth-heterogeneous**, so it requires the **matrix (bimaterial) Riemann path**: in `safs` the gating member `owned_flux_pool_` and `heterogeneous_material_{hpp,cpp}` (Constant/Coefficient modes) already exist, but the `BimaterialFlux` class, the per-face dispatch, and `DepthProfile1D` are **only in hrs-ref** and must be ported (Phases 9–10). The two solvers become an explicit `interior_flux ∈ {scalar, matrix}` choice; **mixed-flux is valid only with scalar** (homogeneous). (c) The **entire friction + state-evolution + native-nucleation kernel is already byte-identical on safs and the reference branch** — so there is **no friction physics to re-derive; that work is wiring, unification, and modularization**. The new physics this pass *adds* is the bimaterial Riemann flux and the depth-profile material, both ported verbatim from `hrs-ref`.

3 Resolved decisions (was: open questions)

3.1 D1 – Fault-basis source → KEEP safs’s; add a planar-TPV correctness test

Keep `safs`’s driver-local per-QP `FaultBasis` (`Compute + AppendSharedFaces + R-005 ComputeQPBasis/ComputeQPBasisShared`, driver `~:1033–:1065`) and the `ref_normal = (0, −1, 0)` convention.

Do **not** adopt the reference branch's `wave.GetFaultBasis()` reuse. Layer in only the empty-fault-rank guard (`GetFaultBasis()` null-check + `static const FaultBasis empty_fault_basis_{}).` **Verification (new):** a unit test asserts that for the planar $y=0$ TPV fault the driver-local per-QP basis yields the canonical frame $\text{dip} = (0, 0, -1)$, $\text{strike} = (+1, 0, 0)$ at every fault QP (Phase 8). This is the “test if planar TPV is correct” the user asked for.

3.2 D2 – CFL safety → adopt opt-in `cfl_safety`, keep BOTH options

Replace `safs's` unconditional `cfl /= 3*(2p+1)` with the reference branch's opt-in `[numerics] cfl_safety ∈ {raw, dg}` (`dg` applies the factor). Set `cfl_safety = "dg"` in **all** configs (TPV and SAFS), which is byte-identical to `safs's` present behaviour for every existing config while exposing `raw` for experiments.

3.3 D3 – Stress-sign convention → two complementary moves

The user's final direction (this pass) is two-part: **(a)** the fault-local pre-stress input is correct *as an option* for the TPV cases — keep it; **(b)** for the **regional projection** (`constant_tensor`), change the input convention so a **positive (right-lateral) input maps to a positive (right-lateral) on-fault** shear, prove the on-fault quantities are unchanged, prove nothing else is affected, and derive the $\text{global} \leftrightarrow \text{local}$ transform with equations. The derivation and proof are in §2-D3.1; the (kept) fault-local option is in §2-D3.2.

D3.1 – Regional projection: right-lateral-POSITIVE input convention (new)

Setup (the $\text{global} \leftrightarrow \text{local}$ transformation). Let S be the regional Cauchy stress tensor (3×3 symmetric) in the SEAS **compression-positive** convention. The fault-local orthonormal frame for the canonical $y=0$ vertical strike-slip fault (from `ref_normal=(0, -1, 0)`, `up=(0, 0, 1)`; confirmed by the D1 planar-basis test) is

```
n = ( 0, -1, 0)  (fault normal)
t1 = ( 0, 0, -1) (dip)
t2 = ( 1, 0, 0)  (strike)
```

The on-fault traction is $T = S \cdot n$ (i.e. $T_r = \sum_c S_{rc} n_c$), resolved in the local frame by the **safs no-flip rule** (`fault/fault_geometry_safs_templated.inl:102–104`, mirrored in `io/field_coefficient.cpp:482–487`):

```
 $\sigma_n = n \cdot T = n^T S n$ 
 $\tau_{\text{dip}}(\tau_1) = t_1 \cdot T = t_1^T S n$ 
 $\tau_{\text{str}}(\tau_2) = t_2 \cdot T = t_2^T S n$ 
```

Substituting the canonical frame (n has only a $-y$ entry, so $T = S \cdot n = (-S_{xy}, -S_{yy}, -S_{yz})$):

```
 $\sigma_n = n \cdot T = (-1)(-S_{yy}) = + S_{yy} = + \sigma_{yy}$   (compression positive)
 $\tau_{\text{dip}} = t_1 \cdot T = (-1)(-S_{yz}) = + S_{yz} = + \sigma_{yz}$   (coefficient +1)
 $\tau_{\text{strike}} = t_2 \cdot T = (+1)(-S_{xy}) = - S_{xy} = - \sigma_{xy}$   (coeff -1 ← inversion)
```

This $\tau_{\text{strike}} = -\sigma_{xy}$ is the rotation artifact, not a bug. Physical check (displacement, not traction-direction): a simple-shear field $u_x = \gamma y$ ($\gamma > 0$) has strain $\epsilon_{xy} = \gamma/2 > 0$, i.e. tension-positive $\sigma_{xy}^{\text{tens}} > 0$; it moves the $+y$ block in $+x$ relative to the $-y$ block — **right-lateral**. In the compression-positive store $S_{xy} = -\sigma_{xy}^{\text{tens}}$, so a **right-lateral** prestress means $S_{xy} < 0$, and

the projection gives $\tau_{\text{strike}} = -S_{xy} > 0$. That is exactly why SAFS today writes $\text{sigma_xy_pa} = -9.89\text{e}6$ to obtain $\tau_{\text{strike}} = +9.89\text{e}6$.

The change. Define the TOML input field sigma_xy_pa to be **right-lateral-positive** (equivalently, the tension-positive Cauchy shear $\sigma_{xy}^{\text{tens}}$), while normal-stress fields stay compression-positive. The TOML→source factory absorbs the rotation’s -1 for the strike off-diagonal **at construction time**:

```
S_xy(stored) = - sigma_xy_pa      # strike off-diag: negate at build
S_yy(stored) = + sigma_yy_pa      # normals: compression-positive (as-is)
S_yz(stored) = + sigma_yz_pa      # dip shear:  $\tau_{\text{dip}} = +\sigma_{yz}$ , sign matches
```

(For the fixed canonical $y=0$ fault used by **every** config, the only off-diagonal whose rotation coefficient is -1 is xy ; yz/xz keep coefficient $+1$, so only sigma_xy_pa is negated. The rule is “negate the components the rotation inverts,” stated per-frame.) The unchanged no-flip projection then yields the user’s requirement:

```
 $\tau_{\text{strike}} = - S_{xy}(\text{stored}) = -(-\text{sigma\_xy\_pa}) = + \text{sigma\_xy\_pa} \quad \checkmark \text{ in==on-fault}$ 
 $\sigma_n = + S_{yy}(\text{stored}) = + \text{sigma\_yy\_pa} \quad (\text{unchanged})$ 
```

Bit-identity proof (same on-fault quantities; nothing else affected). - *Tensor unchanged.* Today: $\text{sigma_xy_pa} = -9.89\text{e}6$, builder stores $S_{xy} = -9.89\text{e}6$. After: $\text{sigma_xy_pa} = +9.89\text{e}6$, builder stores $S_{xy} = -(+9.89\text{e}6) = -9.89\text{e}6$. The **stored Cauchy tensor S is bit-identical**, hence every on-fault scalar (τ_{strike} , τ_{dip} , σ_n , and the derived per-DOF $a/\eta/Dc/V_{\text{init}}$) is bit-identical. - *Projection rule unchanged.* We do **not** touch `fault_geometry_safs_templated.inl` or `field_coefficient.cpp`; the no-flip rule (R-501/R-502, `PLAN_onfaultstress.md:142-164`) is preserved. **Nothing is reverted** — consistent with the CLAUDE.md “don’t revert a fix” rule. The only code change is one sign at the TOML→ConstantTensorStressSource call. - *No other consumer.* **Verified (Agent-1 audit):** the regional StressSource is consumed *only* by `ComputeParams/ProjectFaultPreStress` to seed fault DOFData (`spatial_dyn_driver.cpp:1123-1141`); it is **never** applied as a bulk initial condition and the wave operator never receives it. Negating the input therefore provably cannot affect any bulk/elastic calculation. - *Atomic config update.* The parser negation and the **sign flip of sigma_xy_pa in all SAFS production configs (Dc2/Dc8/Dc10) land together**, so net on-fault stress is unchanged; a golden-value test locks it (below). - *Scope of the convention.* This applies to the hand-authored `constant_tensor` TOML path only. The machine-written **HDF5 sidecar** stays compression-positive (the resolver writes it); `ProjectFaultPreStress` and **T_65_5 (templated == sidecar, fixture $\text{sigma_xy}=0$)** stay green. A documented note warns that expressing the *same* physical state both as a `constant_tensor` TOML and as an HDF5 sidecar now needs opposite sigma_xy signs.

Consistency check before production (new test) — `test_constant_tensor_sign.cpp`: for the SAFS Dc2 stress block (post-flip $\text{sigma_xy_pa} = +9.89\text{e}6$), build the ConstantTensorStressSource via the **factory** (so the negation is exercised), project onto a canonical-frame fault DOF, and assert $\tau_{\text{strike}} == +9.89\text{e}6$ (right-lateral positive), $\sigma_n == +\sigma_{yy}$, $\tau_{\text{dip}} == +\sigma_{yz}$, **bit-identical to the pre-change golden values** captured from the current Dc2 config. Precondition assert: `dip==(0,0,-1), strike==(+1,0,0)`.

D3.2 – Fault-local pre-stress input (kept as an option for the TPV cases)

For TPV205/102/104 the pre-stress is naturally expressed in **fault-local** terms; the user confirmed this is “correct for TPV to use fault-local, this is an option.” Keep it:

```
[stress]
kind      = "fault_local_prestress"
tau_strike_pa = 75.0e6      # right-lateral + (TPV102; 205:70e6 104:40e6)
tau_dip_pa   = 0.0
sigma_n_pa   = 120.0e6      # compression POSITIVE
```

- **The source seeds DOFData directly** — $\text{tau1_0(dip)}=\text{tau_dip_pa}$, $\text{tau2_0(strike)}=\text{tau_strike_pa}$, $\text{sigma_n0}=\text{sigma_n_pa}-P_p$ — **bypassing the Cauchy projection entirely**. This is exactly how the native drivers seed (dynamic/tpv102_setup.hpp:76–78: $\text{d.tau2_0} = \text{TPV102Params::tau_ini}$), so it is **byte-exact to native with no projection round-trip**. No sign flip anywhere; positive $\text{tau_strike_pa} \Rightarrow$ positive $\text{tau2_0} \Rightarrow$ native right-lateral. (*The user confirmed: “we don’t need stress projection for TPV benchmark.”*)
- **Note the harmony with D3.1:** both the kept fault-local input *and* the new constant_tensor convention now make **positive == right-lateral on-fault**, so the two stress paths agree on sign at the TOML surface; they differ only in mechanism (direct seed vs Cauchy projection). A TPV case may use either; the SAFS regional case uses constant_tensor.
- **TPV205** carries positive tau_strike **patches** (background +70 MPa; central +81.6 MPa, left +78 MPa, right +62 MPa) instead of Cauchy sigma_xy patches. The Cauchy ConstantTensorWithPatches source is therefore **not** required for the three primary cases.
- **Implementation note:** FaultGeometry::ComputeParams gains a fault-local seeding overload ComputeParamsFaultLocal(tau_strike, tau_dip, sigma_n, P_p) that assigns the three per-DOF scalars directly; seeding is frame-agnostic (the friction solve still uses the canonical frame for slip *directions* — guarded by the D1 planar-basis test).
- **Consistency check (new test):** test_tpv_toml_stress_sign.cpp builds each TPV FaultLocalPrestressSource from its TOML and asserts $\text{tau2_0} == +\text{tau_strike_pa} == +\tau_{\text{ini_native}}$, $\text{tau1_0} == +\text{tau_dip_pa} == 0$, $\text{sigma_n0} == \text{sigma_n_pa} - P_p$ (native refs: TPV205 70e6 / patch 81.6e6, TPV102 75e6, TPV104 40e6; σ_n 120e6).

3.4 D-rename – general method name → DO the rename + consistency check

Rename FaultGeometry::ComputeSAFSParams → ComputeParams, HasSAFSParams → HasParams, member safs_params_computed_ → params_computed_ across fault/fault_geometry.hpp, fault/fault_geometry_safs.inl, fault/rate_state_fault.hpp, config/seas_config.hpp, config/seas_config_bridge.hpp, spatial/code/spatial_stress.{hpp,cpp}, and tests/unit/test_compute_safs_params.cpp. The fault kind string stays informational (no code branches on it). **Consistency check before production:** after the rename the full unit suite (esp. test_compute_safs_params) plus the new test_tpv_toml_stress_sign must be green, and a symbol grep must show zero remaining ComputeSAFSParams/HasSAFSParams references (Phase 1 acceptance).

4 Byte-exactness contract (the non-negotiable)

The verified standalone drivers `drivers/tpv{205,104,102}_driver.cpp` are the oracle. **No function is approximated.** Verified by diff (`/tmp/branchcmp/pass2_exactness.md`): the kernel files `friction/{dieterich_ruina,state_evolution,slip_law_srw_psi, friction_coeff_stable}.hpp`, all `config/tpv*_params.hpp`, all `dynamic/tpv*_setup.hpp`, and `dynamic/tpv10{2,4}_nucleation.hpp` are **byte-identical on safs and the reference branch**. Therefore the modules below *call* these existing functions; they do not reimplement them.

Case	Friction-law solve (canonical fn)	State evolution	Nucleation (canonical fn)	Unified-iterator mapping (§5.1)
TPV205 (LSW)	<code>SolveLSW_TPV205</code> + <code>LSWFrictionCoefficient_TPV205</code> (<code>dynamic/tpv205_friction.hpp:54,94</code>), barrier sentinel $\mu_s=1e4$	none	static IC — <code>ComputeTau2_0_TPV205</code> from the (<code>config/tpv205_params.hpp:150,189</code>), seed <code>tau2_0/lsw_mu_s</code> ; spatial uses fault-local prestress (D3.2) or <code>constant_tensor</code> (D3.1), seeded directly	<code>LinearSlipWeakeningIterator</code> — the <code>TPV205SubStepIterator::Step0</code> closed-form body verbatim (<code>SolveLSW_TPV205</code> , callback nucleation)
TPV102 (RS aging)	<code>DieterichRuina::SolveSlipRatePsi</code> (Brent, <code>friction/dieterich_ruina.hpp:322,470</code>)	<code>SlipLawSRWPsi</code> + <code>UpdateStateAnalytic</code> (<code>friction/tpv102_state.hpp:177,304</code>)	compact <code>Psell</code> <code>Fd(r)=exp(r²/(r²−R²))</code> policy is (<code>dynamic/tpv102_nucleation.hpp:100</code>) ≡ spatial <code>ResolveGradualOverideCircular/Apply..</code> (math-identical, proven)	<code>RateStateSubStepIterator<RateStateAnalytic></code> — policy is <code>UpdateStateAnalytic(...)</code> ; (<code>dynamic/tpv102_nucleation.hpp:100</code>) sequence identical to <code>Tpv102SubStepIterator</code>
TPV104 (RS SRW)	same Brent chain	<code>SlipLawSRWPsi</code> + <code>UpdateStateAnalytic</code> (<code>friction/slip_law_srw_psi.hpp:92,100</code>), per-QP V_w	same compact <code>SlipLawSRW</code> (<code>friction/slip_law_srw_psi.hpp:92,100</code>), per-QP V_w	<code>RateStateSubStepIterator<RateStateAnalyticSlipLawSRW></code> — policy line is <code>UpdateStateAnalyticSlipLawSRW</code> <code>V_w[i], ...)</code> ; V_w span owned by the iterator

Case	Friction-law solve (canonical fn)	State evolution	Nucleation (canonical fn)	Unified-iterator mapping (§5.1)
TPV31 (LSW, het)	SolveLSW chain + depth-linear cohesion C0(depth)	none	instantaneous circular cosine-tapered overstress (hrs-ref tpv31.toml:45– 50)	LinearSlipWeakeningIterator (same as TPV205) on the matrix Riemann path (§5.3); nucleation = InstantaneousOverstressCircular

Shared infrastructure (must NOT be duplicated by any per-law module): ADER predictor `WaveOperator::ComputeADERSubStepStates + EvaluateBulkAtFaultQPsCanonical`; the driver helper `AdvanceADERWithSubStep_Spatial`; `FaultFaceFlux::`{`ComputeTrialTraction`, `ComputeStageState`, `BuildImposedState`, `WriteBackState`}; `FaultBasis::ComputeOrientedFrame`; per-DOF η/Z impedances; `SpatialFrictionResolver`; and the new `SubStepIteratorBase::RunSubSteps_` skeleton (§5.1) shared by every iterator.

Deviations to fix (wiring/unification + the bimaterial/material port; zero friction-physics reimplementation): 1. **Unify the iterators (§5.1).** Create one `dynamic/friction_substep_iterator.{hpp,cpp}` holding `SubStepIteratorBase`, `RateStateSubStepIterator<StatePolicy>`, and `LinearSlipWeakeningIterator`, plus `friction/state_policies.hpp`. The unified RS iterator has the per-sub-step `nuc_callback` built in (no need to retrofit the old iterators). The standalone `tpv{205,102,104}_substep_iterator.{hpp,cpp}` are **left untouched as the byte-exact oracle**; parity tests prove the unified iterator reproduces them bit-for-bit. Retiring the old iterators is a deferred follow-up. 2. Add the additive `GradualOverstressCompactCircular` and `InstantaneousOverstressCircular` resolver/applicators to `dynamic/spatial_nucleation.{hpp,cpp}` (additive, 0 deletions). 3. D3 (both moves): add `FaultLocalPrestressSource + ComputeParamsFaultLocal` (D3.2, direct seed, no projection), **and** the right-lateral-positive `constant_tensor` factory convention (D3.1, one sign at construction; stored tensor bit-identical; projection rule untouched). 4. TPV104 V_w boxcar reproduced by the `[friction.rate_state]` TOML (config-level). 5. **Port the matrix Riemann path** (`dynamic/godunov_flux_bimaterial.{hpp,cpp}` + the `owned_flux_pool_dispatch + ExchangeBiMaterialNeighbours_`) and the **DepthProfile1D material** from hrs-ref (Phases 9–10) — these are *new physics*, copied verbatim from the reference implementation, not reimplemented.

5 Git provenance, reference, and existing scaffolding

- The reference TOML workflow lives as **unpushed commits in a separate clone**: `/Users/chunhuizhao/projects/feature/heterogeneous_riemann_solver`, HEAD `e30d003`. It is *not* on origin (origin’s same-named branch is the merge-base `b18bed5`, an ancestor of `safs`).
- True merge-base of the two working lines: `0e13b25`. In this repo the reference tip is fetched as tag **hrs-ref**. Diffs reproducible via `git diff 0e13b25 hrs-ref -- <path>` (reference side) / `git diff 0e13b25 safs -- <path>` (safs side).

- Per-area analysis archived at /tmp/branchcmp/agent{1..5}*.md (pass 1) and /tmp/branchcmp/pass2_{exact} (pass 2).

5.1 RS scaffolding already in safs (the SAFS+RS head start)

A source audit (this pass) confirms the SAFS + aging rate-and-state goal is a **driver-wiring** job, not a build-the-module-system job — almost every piece exists and is reusable:

Piece	Status in safs	Location
[friction.rate_state] parse + RateStateBlock + [meta].law=rate_state dispatch	EXISTS	spatial_friction.{hpp:241–253, cpp:388–466,795–818}
ResolveRateState(...) (ParMesh + serial; takes pp + sigma_n_total_per_dof) + RateStatePerDOFParams	EXISTS	spatial_friction.{hpp:293–326, cpp:963–1118}
InitializeFaultDOFs_Spatial_RS (seeds a/Dc/impedance/pre-stress; psi=0 stub)	EXISTS	spatial_setup.hpp:301–346
Kernels DieterichRuinaFriction::InitializePsi, AgingLawPsi, UpdateStateAnalytic	EXISTS (byte-identical to native)	friction/{dieterich_ruina.hpp:506, state_evolution.hpp:180,304}
Aging RS sub-step iterator Tpv102SubStepIterator(FaultFaceFlux&, const AgingLawPsi&)	EXISTS	dynamic/tpv102_substep_iterator.hpp:73
FaultFrictionLaw::RateAndState wave-op dispatch (EvaluateADER on interior+shared faces)	EXISTS	dynamic/wave_operator.hpp:79,254–264
Gaussian gradual_overstress nucleation (law-agnostic; writes tau{1,2}_nuc)	EXISTS	dynamic/spatial_nucleation.{hpp:145–174, cpp:80–178}

Missing (the only RS work, all in Phases 1–3): the driver hard-rejects rate_state (spatial_dyn_driver.cpp:716), never calls ResolveRateState/InitializeFaultDOFs_Spatial_RS, hard-types its sub-step machinery to the LSW Tpv205SubStepIterator (:367,:1736), hard-codes SetFaultFrictionLaw(LSW) (:971), has **no equilibrium- ψ seed** (psi=0 stub at spatial_setup.hpp:341), and Tpv102SubStepIterator lacks a nuc_callback overload + SetDiagNumLocalFaultQPs. That is the entire gap, and it is what Phases 1–3 close.

6 Module architecture (the “parts”)

Chosen shape (simplified this pass): three orthogonal “parts,” each a small strategy interface with a one-line factory. The driver carries no friction if/else, no nucleation switch, and no flux if/else — it holds three `unique_ptrs` and calls through interfaces. Adding a friction law, a nucleation method, or a flux variant is a new class + one factory line, never a driver edit. The three parts are **method-oriented**, not benchmark-oriented:

```
friction_laws      : IFrictionIterator    <- {LSW, RS-aging, RS-slip-law-SRW}
nucleation_methods : INucleationMethod   <- {Gaussian, CompactCircular,
                                             InstantaneousCircular, Static}
riemann_flux      : interior_flux        <- {scalar (homogeneous),
                                             matrix (heterogeneous/bimaterial)}
```

6.1 friction_laws part (unified, method-oriented)

Phasing (see §7): to land the SAFS+RS goal fast and on proven code, **Phases 1–3 wire a thin IFrictionIterator strategy with adapters over the existing Tpv205/TPV102 iterators** (`LswFrictionIterator`, `RateStateAgingFrictionIterator`). **Phase 5** then introduces the unified design below (`SubStepIteratorBase` + `RateStateSubStepIterator` <Policy> + `LinearSlipWeakeningIterator`, adding the SRW law) and re-points the factory to it, parity-tested bit-for-bit against the standalone oracle. The `IFrictionIterator` interface and the driver are stable across both; only the iterator *internals* change.

The end-state collapses the three benchmark-named sub-step iterators into a method-oriented set sharing one skeleton. The structural cleavage is **LSW vs rate-state**, not 102-vs-104: TPV102 and TPV104 differ in *exactly one line* (the state-variable update), while TPV205 has a different kernel *shape* (closed-form, no root-find, no state). So:

New file `dynamic/friction_substep_iterator.{hpp,cpp}` holds:

- **SubStepIteratorBase** — the common ~250-line ADER sub-step envelope that is byte-for-byte identical across all three today (the `SetSubSteps` validator, the $\Sigma \Delta T == dt_macro$ tolerance, the `memset` of `I_imp_*`, the per-(sub-step, QP) $Q_{\pm}$ build for both the time-averaged I/dt and per-sub-step $Q_{pointwise}$ cases, the `accum_scale` accumulation, the slip accumulation $d.slip\{1,2\} += s.V\{1,2\} * dt_sub$, and `WriteBackState` on the last sub-step). It exposes a single templated driver `RunSubSteps_(dofs, coords, qsource, dt_macro, t0, I_imp_p, I_imp_m, nuc_fn, step_fn)` that calls a per-QP `step_fn` functor and a `nuc_fn(t_sub_end, dt_sub)` once per sub-step before the QP loop. *This collapses both the triplication and the Advance vs AdvanceWithSubStepStates duplication.*
- **RateStateSubStepIterator<StatePolicy>** — the rate-state iterator, parameterized by a state-evolution policy. Its per-QP `step_fn` is the existing token sequence `flux_.ComputeStageState(d, Qp, Qm, s, method) → d.slip{1,2} += s.V{1,2} * dt_sub → d.psi = StatePolicy::UpdatePsi(law_, d, s.V_abs, dt_sub, extra, i) → flux_.BuildImposedState(...)`. The policy is the *only* variation point.

```

template <class StatePolicy>
class RateStateSubStepIterator : public SubStepIteratorBase,
                                public IFrictionIterator {
public:
    RateStateSubStepIterator(FaultFaceFlux& flux,
                             const typename StatePolicy::Law& law);
    // IFrictionIterator::Advance(...) forwards to RunSubSteps_ with the
    // policy step_fn; FrictionSolver::Method + ExtraState come from cfg.
private:
    const typename StatePolicy::Law& law_;
    typename StatePolicy::Extra      extra_; // std::monostate for aging;
                                         // owns V_w span for SRW
};

```

- **LinearSlipWeakeningIterator** — a separate small class (no method, no ComputeStageState, no state variable). Its step_fn lifts Tpv205SubStepIterator:: StepOneQP_ **verbatim** (ComputeTrialTraction + SolveLSW_TPV205 + the [SLIP] diagnostic + callback nucleation). Forcing LSW into the RS template would need dead policy hooks, so it stays distinct — but still inherits SubStepIteratorBase for the envelope.

State policies live next to their laws in friction/state_policies.hpp (so the friction-method coupling sits with the friction code):

```

struct RateStateAgingPolicy {                // TPV102
    using Law    = AgingLawPsi;
    using Extra  = std::monostate;
    static real_t UpdatePsi(const Law& L, const DOFData& d, real_t V,
                             real_t dt, const Extra&, int /*i*/) {
        return UpdateStateAnalytic(d.psi, V, d.Dc, dt,
                                     L.GetF0(), L.GetB(), L.GetV0());
    }
};

struct RateStateSlipLawSrwPolicy {           // TPV104
    using Law    = SlipLawSRWPsi;
    using Extra  = std::span<const real_t>; // V_w, owned by iterator
    static real_t UpdatePsi(const Law& L, const DOFData& d, real_t V,
                             real_t dt, const Extra& Vw, int i) {
        return UpdateStateAnalyticSlipLawSRW(d.psi, V, d.Dc, dt, Vw[i],
                                                d.a, L.GetB(), L.GetV0(), L.GetF0(), L.GetMuW());
    }
};

using RateStateAgingIterator =
    RateStateSubStepIterator<RateStateAgingPolicy>;
using RateStateSlipLawSrwIterator =
    RateStateSubStepIterator<RateStateSlipLawSrwPolicy>;

```

The driver-facing strategy interface and factory:

```
class IFrictionIterator {
public:
    virtual void SetSubSteps(std::vector<real_t> dT, std::vector<real_t> w) = 0;
    virtual std::vector<real_t> GetDeltaT() const = 0;
    virtual std::vector<real_t> GetTimeWeights() const = 0;
    virtual void Advance(std::vector<DOFData>& dof,
        const std::vector<Vector>& coords,
        const std::vector<std::vector<real_t>>& Qp,
        const std::vector<std::vector<real_t>>& Qm,
        real_t dt, real_t t0, real_t* Iimp_p, real_t* Iimp_m,
        const std::function<void(real_t,real_t)>& nuc_cb) = 0;
    virtual void SetDiagNumLocalFaultQPs(int n) = 0; // safs speckle diag hook
    virtual FaultFrictionLaw WaveOpLaw() const = 0; // LSW | RateAndState
    virtual ~IFrictionIterator() = default;
};

// dynamic/friction_iterator_factory.{hpp,cpp}
std::unique_ptr<IFrictionIterator> MakeFrictionIterator(
    const spatial::SpatialFrictionConfig& cfg, FaultFaceFlux& flux,
    const spatial::RateStatePerDOFParams* rs /*nullptr for LSW*/);
// is_lsw -> LinearSlipWeakeningIterator
// RS && aging_law -> RateStateAgingIterator
// RS && slip_law_srw -> RateStateSlipLawSrwIterator
```

Byte-exactness guarantee. The unified iterators are *new code*; the standalone `tpv{205,102,104}_substep_iterator` stay **untouched as the oracle**. The kernel call tokens are not reordered — the policy only *selects* the `UpdateState...` call that the original code already inlined. Parity tests (`tests/unit/test_tpv104_substep_iterator` and siblings) plus a new RS/LSW one-shot parity assert the unified iterator reproduces each standalone iterator bit-for-bit on a fixed fixture. Diagnostics (`SEAS_DIAG_TPV104_STATE`, `slip_rate_substep_max`) stay behind their `#ifdef/env` gates. *Net: ~250 triplicated lines → one RunSubSteps_; 102 vs 104 → one policy line; 205 stays a focused closed-form class; names are method-oriented.*

6.2 nucleation_methods part

New file `dynamic/nucleation_method.hpp` — interface:

```
class INucleationMethod {
public:
    virtual void ApplyIncrement(std::vector<DOFData>& dof,
        real_t t, real_t dt) = 0;
    virtual void ApplyOnce(std::vector<DOFData>& dof) {} // static/instantaneous
    virtual bool IsPerSubStep() const = 0;
    virtual ~INucleationMethod() = default;
};
```


Concrete methods (all method-oriented, benchmark-agnostic): GaussianGradualOverstress (wraps safs's existing ResolveGradualOverstress/ ApplyGradualOverstressIncrement — SAFS + as a TPV option), CompactCircularGradualOverstress (the new SCEC bell $F(r)=\exp(r^2/(r^2-R^2))\cdot\text{smoothStep}$ — TPV102/104), InstantaneousOverstressCircular (cosine-tapered one-shot patch — **TPV31**; ApplyOnce seeds tau_nuc, ApplyIncrement no-ops), and StaticOverstress (TPV205 — patches already in tau_pre_; both hooks no-op). ApplyIncrement is the per-sub-step nuc_cb body the driver passes into IFrictionIterator::Advance.

```
// dynamic/nucleation_factory.{hpp,cpp}
std::unique_ptr<INucleationMethod> MakeNucleation(
    const spatial::SpatialFrictionConfig& cfg,
    const std::vector<Vector>& dof_coords,
    const std::vector<FaultBasisRow>& dof_basis);
```

6.3 riemann_flux part (homogeneous scalar vs heterogeneous matrix)

The interior (non-fault) face flux becomes an explicit, config-selected choice. **The fault flux is unaffected** — FaultFaceFlux is used on fault faces regardless of the interior choice (verified: the bimaterial path in hrs-ref applies only to interior non-fault faces; the fault still uses FaultFaceFlux). So this part is orthogonal to friction_laws.

- **interior_flux = "scalar"** (homogeneous Godunov, the existing safs path): the scalar WaveOperator(mesh, order, λ , μ , ρ , bc) ctor; owned_flux_pool_ == nullptr; the per-face dispatch uses the single shared flux_. **Mixed-flux (mixed_flux $\in$ {adjacent,...}, Zhang et al. 2023) is valid ONLY here** (a per-face Central-vs-Interior choice on the scalar path).
- **interior_flux = "matrix"** (heterogeneous / bimaterial Riemann): the heterogeneous WaveOperator(mesh, order, const MaterialField&, bc) ctor builds owned_flux_pool_ per-element (material.EvalAt(elem,...)), precomputes per-(face,side) flux matrices via BimaterialFlux::BuildPerFaceFluxMatricesGlobal, and at runtime applies BimaterialFlux::ApplyPerFaceFluxNeighbor, Q_self, Q_nbr, F_h_self) at each interior QP. mixed_flux MUST be unset (mutual-exclusion guard).

Port required (hrs-ref → safs, Phase 9), verbatim: dynamic/godunov_flux_bimaterial.{hpp, cpp} (the BimaterialFlux class, SeisSol positive-stress matrix convention), the owned_flux_pool_ dispatch branches in wave_operator.inl (Mult + ADER interior-face paths), BuildPerFaceBimaterialFluxMatrices, the cross-rank ExchangeBiMaterialNeighbours_ + shared_face_neighbour_material_, and per_elem_lmr_/MaxCpInElement CFL caching. In safs the gating member owned_flux_pool_ and heterogeneous_material.{hpp, cpp} (Constant/ Coefficient) **already exist** — only BimaterialFlux and the dispatch wiring are missing. The depth-dependent MaterialField (DepthProfile1D / MakeDepthProfile1DMaterial, Coefficient mode) is ported in Phase 10 for TPV31.

```
// In the driver (Phase 5/9): one branch, no per-face logic in the driver.
std::unique_ptr<WaveOperator> wave_ptr;
if (cfg.numerics.interior_flux == InteriorFlux::Scalar)
    wave_ptr = std::make_unique<WaveOperator>(pmesh, order, lam, mu, rho, bc);
else /* Matrix */
```



```

    wave_ptr = std::make_unique<WaveOperator>(pmesh, order, material_field, bc);
WaveOperator& wave = *wave_ptr;

```

6.4 Shared-rank correctness constraints (must hold for every fault-flux / exchange change)

The unified friction iterator and the new matrix-flux path both touch cross-rank fault and interior faces. From the shared-fault audit (R-001/R-004/R-101/R-1303/R-1601), the design **must**:

1. **Detect shared fault faces symmetrically across ranks** — gate on GetSharedFaceTransformations/an Allgather of global face IDs, **never** on local boundary-element ownership (R-001: one-sided detection silently dropped RHS → 6% error at np=400). The unified iterator changes *nobody's* face detection; it must keep using the existing symmetric fault_shared_tagged_set.
2. **Keep the R-1601 shared-QP rule intact.** Shared fault QPs **bypass the sub-step iterator and run inline EvaluateADER** in ComputeADERSHaredFaceFluxRHS; only interior fault QPs go through the iterator. The unification must preserve this exact split — RunSubSteps_ operates on the interior-QP set; the shared path is untouched. (At np>1 a rank-local canonical frame in the iterator caused tau=4e28 → SIGABRT within ~7 macro-steps.)
3. **Exchange neighbour bulk Q before reading it** on shared QPs (ghost_gf_full_state_→ExchangeFaceNbrData and map components via GetFaceNbrElementVDofs — **byNODES, not contiguous slabs** (R-004). Per-sub-step ghost coverage is mandatory (R-1303): the predictor is element-local and does not fill ghosts.
4. **Reconcile friction DOFData across the seam before flux assembly** — the boss rank (lower ID) broadcasts {psi, slip1/2, v1/2, tau*_corr, sigma_n_corr, slip_rate} and the canonical imposed state via ExchangeAndPairSharedFaultQPs; the peer overwrites locally so both assemble bit-identical flux. **For rate-state this is the open shared-fault-RS ψ risk** (the audit's R-001/R-004 lineage; = the review's R-004): the reconcile **does** broadcast ψ (payload[5]), so cross-rank ψ is single-valued — the residual issue is *temporal* (end-of-step ψ on the inline shared flux → 1st-order at ader_order $\geq$ 2), gated in **Phase 3** (the SAFS+RS deliverable), not Phase 7. (*The safs-audit "R-001/R-004" here are the shared-fault debug-history findings — a separate numbering from the REVIEW.md R-001...R-008.*)
5. **Honour the collective contract** — every rank with *any* shared face (fault or not) must call the exchange, gated on pmesh.GetNSharedFaces() > 0, NOT on having shared *fault* faces (R-1600: a missing call deadlocked for 13.5 min). The new bimaterial ExchangeBiMaterialNeighbours_ adds one more collective and must obey the same gate.
6. **Use absolute QP indexing** on the shared branch (R-1304), and keep shared fault QPs off the precomputed-flux table (they always run inline so the reconciliation payload is complete).
7. **MPI parity is a gate, not an afterthought** — Phases 3/9 add a forced-2-rank single-shared-fault-face test asserting cross-rank DOFData symmetry and flux bit-identity.

6.5 What leaves vs stays in the driver

Leaves (into the factories/iterators): the three-way iterator construction, AgingLawPsi/SlipLawSRWPsi construction, the nucleation switch/nuc_cb lambda, the equilibrium- ψ seed loop, and the scalar-vs-matrix WaveOperator ctor branch (one if). **Stays (safs fixes — DO NOT MOVE)**:

R-005 per-QP basis (BuildPerDOffFaultTables, ComputeQPBasis*), driver-local FaultBasis + ref_normal=(0,−1,0), R-101 shared-fault tripwires + SEAS_R101_NONFATAL + SEAS_DIAG_BLOWUP + slip_rate_substep_max reset + NaN tripwire, speckle/Phase-1 diag, ParaView fault VTU plumbing (_k4, WriteFaultSurfaceVTU, SetRegisterFaultProjectionsInVolumePV), checkpoint dedup R-011, the R-1601 shared-QP inline bypass, and the AdvanceADERWithSubStep_Spatial body — only its iterator parameter changes from Tpv205SubStepIterator& to IFrictionIterator&; SetDiagNumLocalFaultQPs is a virtual.

7 Code graph – full structure

Five views of the end-state structure. Status legend: [NEW] create, [MOD] modify (preserve existing behaviour), [REUSE] use as-is (no edit), [PORT] copy verbatim from hrs-ref, [ORACLE] keep untouched as the byte-exact reference. The three “parts” — **friction_laws**, **nucleation_methods**, **riemann_flux** — live under dynamic/ (G1) and are exercised in G2/G3/G5.

7.1 G1 – Code map (files by role)

```
miniapps/seas/
|- drivers/spatial_dyn_driver.cpp [MOD] orchestrator; safs fixes stay
|- spatial/code/
|  |- spatial_friction.{hpp,cpp} [MOD] TOML schema + resolver
|  `-- spatial_stress.{hpp,cpp} [MOD] +FaultLocalPrestress; ct sign
|- dynamic/ ( friction_laws + nucleation_methods + riemann_flux )
|  |- friction_substep_iterator.* [NEW] base + RS template + LSW
|  |- friction_iterator.hpp [NEW] IFrictionIterator + factory
|  |- friction_iterator_factory.* [NEW] MakeFrictionIterator(cfg,flux,rs)
|  |- nucleation_method.hpp [NEW] INucleationMethod + 4 methods
|  |- nucleation_factory.* [NEW] MakeNucleation(cfg,coords,basis)
|  |- spatial_nucleation.{hpp,cpp} [MOD] +CompactCircular +InstCircular
|  |- tpv205/102/104_substep_iter* [ORACLE] standalone-driver reference
|  |- spatial_setup.hpp [MOD] + RS IP-aware init overload
|  |- wave_operator.{hpp,inl} [MOD] + matrix dispatch; scalar same
|  |- fault_face_flux.{hpp,cpp} [REUSE] trial/stage/imposed/writeback
|  |- godunov_flux_bimaterial.* [PORT] BimaterialFlux (matrix Riemann)
|  `-- heterogeneous_material.* [MOD] +DepthProfile1D (Coefficient)
|- friction/ ( law kernels, byte-identical to native )
|  |- dieterich_ruina.hpp [REUSE] Brent solve + InitialStatePsi
|  |- state_evolution.hpp [REUSE] AgingLawPsi + analytic update
|  |- slip_law_srw_psi.hpp [REUSE] SlipLawSRWPsi + analytic update
|  `-- state_policies.hpp [NEW] Aging/SlipLawSRW psi policies
|- fault/
|  |- fault_basis.hpp [REUSE] oriented frame (R-005; D1)
|  |- fault_geometry.hpp/_safs.inl [MOD] rename + ComputeParamsFaultLocal
|  `-- rate_state_fault.hpp [MOD] rename knock-on
```

```

|- config/seas_config*.hpp          [MOD] rename knock-on
|- tpv{205,104,102}/configs/*.toml [NEW] stress + friction + nucleation
|- tpv31/ (config,mesh,bench,tests) [PORT] het-material LSW case
|- safs/.../config/*rate_state*     [NEW] SAFS rate-state config
|- jobs/{tpv*_spatial,safs}/*.sbatch [NEW] run scripts (dev + 8-rank)
`- tests/unit/test_*.cpp            [NEW] sign/config/stress/factory/parity

```

7.2 G2 – Runtime wiring (setup: config -> objects)

TOML (--config X.toml)

```

| spatial::LoadSpatialFrictionConfig()
v

```

SpatialFrictionConfig cfg -- read by:

```

|- [meta].law / [friction].state_evolution -> is_lsw, rs_use_srw
|- [stress].kind -> stress seeding (see G4)
|- [friction.*] -> Resolve{SlipWeakening|RateState} -> per-DOF params
|- [nucleation] -> MakeNucleation(cfg,coords,basis)
| -> INucleationMethod {Gaussian|CompactCircular|InstCirc|Static}
|- [material].kind -> Constant | DepthProfile1D (-> MaterialField)
|- [numerics] -> interior_flux {scalar|matrix}, mixed_flux,
| cfl_safety, fault_iterator
v

```

WaveOperator ctor (see G5):

```

| scalar -> WaveOperator(mesh,p,lam,mu,rho,bc) owned_flux_pool_=0
` matrix -> WaveOperator(mesh,p,MaterialField,bc) builds flux pool
v

```

DOFData[] (tau1_0, tau2_0, sigma_n0, a, Dc, psi, ...)

```

| (RS only) SeedEquilibriumPsi_RS -> psi = InitialStatePsi(...)
v

```

MakeFrictionIterator(cfg, fault_flux, rs?) -> unique_ptr<IFrictionIterator>

```

| LinearSlipWeakeningIterator (LSW)
| RateStateSubStepIterator<RateStateAgingPolicy> (aging)
| RateStateSubStepIterator<RateStateSlipLawSrwPolicy> (SRW, owns V_w)
v

```

wave.SetFaultFrictionLaw(fr->WaveOpLaw()) // LSW | RateAndState

7.3 G3 – Per-macro-step call & data flow (time loop)

for each macro step dt: [driver loop -- safs fixes live HERE]

AdvanceADERWithSubStep_Spatial(wave,*fr,dof,coords,Q,dt,...,nuc_cb)

```

|- wave.ComputeADERSubStepStates(...) -- SHARED predictor
| (matrix path: per-element CK via ApplyJacobianPerElementDOF_)
|- wave.EvaluateBulkAtFaultQPsCanonical(...) -- SHARED (R-005;D1)
`- fr->Advance(...) -> SubStepIteratorBase::RunSubSteps_ [INTERIOR QPs]
    ` per ADER sub-step:
        |- nuc_cb(t,dt) -> INucleationMethod::ApplyIncrement

```

```

|         writes DOFData.tau{1,2}_nuc (nucleation_methods)
|- FaultFaceFlux::ComputeTrialTraction/ComputeStageState -SHARED
|- friction: LSW closed-form | RS Brent      (friction_laws)
|- StatePolicy::UpdatePsi: none|Aging|SlipLawSRW
`- FaultFaceFlux::BuildImposedState/WriteBackState -- SHARED

V
wave.ComputeADERSHaredFaceFluxRHS(...) [SHARED-FAULT QPs, R-1601]
  `- inline EvaluateADER (NOT the iterator) + boss-rank DOFData reconcile
wave.AdvanceADER(... imposed states ...) -- SHARED apply
  ` (matrix path: BimaterialFlux::ApplyPerFaceFlux on interior faces)
post-step (UNCHANGED): R-101 tripwire | NaN check | slip_rate_substep_max
reset | station write | checkpoint R-011 | fault VTU

```

7.4 G4 – Stress-source dispatch (both paths: positive input = right-lateral on-fault)

```

[stress].kind
|- "fault_local_prestress" (TPV205/102/104 option; D3.2)
|   -> ComputeParamsFaultLocal(tau_strike, tau_dip, sigma_n, P_p)
|   -> tau1_0=tau_dip ; tau2_0=+tau_strike ; sigma_n0=sigma_n-P_p
|   (direct seed; NO Cauchy tensor, NO projection; == native)
`- "constant_tensor" (SAFS regional; TPV option; D3.1)
    factory: S_xy = -sigma_xy_pa (negate strike off-diag at build)
             S_yy = +sigma_yy_pa (normals compression-positive)
    -> ComputeParams<ConstantTensorStressSource> (no-flip, UNCHANGED)
    -> sigma_n+=S_yy ; tau_dip+=S_yz ; tau_strike=-S_xy+=sigma_xy_pa
        (stored tensor BIT-IDENTICAL to today; on-fault unchanged)

```

Both paths now satisfy **positive input** $\Rightarrow$ **right-lateral-positive on-fault**. The `fault_local` path seeds directly (no projection). The `constant_tensor` path keeps the no-flip projection rule verbatim; only the TOML $\rightarrow$ source factory negates the strike off-diagonal, so the stored Cauchy tensor and every on-fault quantity are bit-identical to the current SAFS configs (proof in §2-D3.1). The regional `StressSource` is fault-only — it seeds `DOFData` and is never a bulk initial condition — so the change is provably local.

7.5 G5 – Riemann-flux dispatch (scalar vs matrix; fault flux unaffected)

```

[numerics].interior_flux
|- "scalar" (TPV205/102/104, SAFS) -- homogeneous Godunov
|   WaveOperator(mesh,p,lam,mu,rho,bc); owned_flux_pool_ = nullptr
|   interior face: flux_.Interior(...) [+ optional mixed Central]
|   (mixed_flux ALLOWED here only)
`- "matrix" (TPV31) -- heterogeneous/bimaterial
    WaveOperator(mesh,p,MaterialField,bc); builds owned_flux_pool_
    precompute per-(face,side) via BuildPerFaceBimaterialFluxMatrices_
    cross-rank: ExchangeBiMaterialNeighbours_ (collective; §5.4)
    interior face: BimaterialFlux::ApplyPerFaceFlux(...)
    (mixed_flux FORBIDDEN; mutual-exclusion guard)

```

FAULT faces: FaultFaceFlux + friction (UNCHANGED on both paths)

The Riemann choice changes only **interior non-fault** faces; fault faces always use FaultFaceFlux + the friction iterator, so friction_laws and riemann_flux are orthogonal.

8 Master phase / file table

Prioritization (this pass): the SAFS + aging rate-and-state run is the near-future goal, and the **entire RS config/resolver/setup/kernel/iterator/wave-op scaffolding already exists in safs** (verified — see §4.1). So **Phases 1–3 deliver a working SAFS+RS run** by wiring the existing pieces (driver was hard-gated to LSW); Phases 4–10 add the method-oriented unification, the TPV regression, the matrix Riemann solver, and TPV31. The » marks the near-future deliverable.

Phase	Files	Action
1 RS primitives »	dynamic/tpv102_substep_iterator.cpp dynamic/spatial_setup.hpp (+dieterich_ruina.hpp include), Makefile; new tests/unit/test_tpv102_nuc_case.cpp test_seed_equilibrium_psi_rs.cpp	add(Tpv102SubStepIterator nuc_callback overload + SetDiagNumLocalFaultQPs (additive, byte-safe); add SeedEquilibriumPsi_RS R-001 d.eta_s+V_init>0; R-009 V_0==FrictionSolver::V0 guard); link tpv102 iterator + test targets
2 Friction strategy »	new dynamic/friction_iterator.hpp friction_iterator_factory.hpp modify drivers/spatial_dyn_driver.cpp (retype :367 + R-010 rename :439 call→Advance); new tests/unit/test_friction_iterator.cpp test_advance_interface_compiler.cpp	IFrictionIterator (incl. R-010 setPlat/GetTimeWeights) + Lsw/RateStateAging adapters over the existing Tpv205/Tpv102 iterators; MakeFrictionIterator (R-009 defensive V0 guard); AdvanceADERWithSubStep→IFrictionIterator LSW byte-unchanged

Phase	Files	Action
3 SAFS+RS run »	drivers/spatial_dyn_driver.cpp spatial/code/spatial_friction (R-006); SAFS configs (Dc2/8/10 sigma_xy_pa flip, atomic); new RS config + sbatch + test_constant_tensor_sign.cpp (parametrized), test_resolve_rate_state_guard test_paraview_state_channel_rej test_shared_fault_rs_psi_consts	remove is_lsw reject (:716); gated RS branch (R-002: outer-scope lsw+rs, slip_weakening.has_value() under is_lsw) ResolveRateState(R-003 PorePressureSpec{})+InitializeFaultDOFs SetFaultFrictionLaw(RateAndState); Paraview state→ψ (R-005); rejection per-DOF b/V ₀ /f ₀ (R-006); n _{pl2} cap→allow a>b (R-011); reuse Gaussian nucleation; D3.1 factory negation → » WORKING SAFS+RS
4 D-rename	fault/fault_geometry.hpp/_saf rate_state_fault.hpp, config/seas_config*.hpp, spatial/code/spatial_stress.{hpp,cpp} tests/unit/test_compute_safs_params.cpp	computeSAFSParams→ComputeParams/HasPara grep-zero residual; suite green (cosmetic generalization, no behavior change)
5 Unified iterator	new dynamic/friction_substep_iterator. friction/state_policies.hpp; modify friction_iterator_factory.cpp new test_friction_substep_iterator_parity.cpp	SubStepIteratorBase+RateStateSubStepIt {hpp,cpp}, ing+SRW)+LinearSlipWeakeningIterator; re-point the Phase-2 adapters; bit-parity vs the standalone oracle
6 TPV config + D3.2	spatial/code/spatial_frictionTPV, n _{pl2} cap spatial_stress.{hpp,cpp}, dynamic/spatial_setup.hpp	TPV, n _{pl2} cap/flux vocabulary (interior_flux∈{scalar,matrix}, material.kind, SRW state_evolution, boxcar_taper); FaultLocalPrestressSource+ComputeParams (D3.2)
7 nucleation_methods	new dynamic/nucleation_method.hpp nucleation_factory.{hpp,cpp}; modify dynamic/spatial_nucleation.{hpp,cpp}	INucleationMethod+factory; add compact-circular and instantaneous-circular resolvers

Phase	Files	Action
8 TPV benchmarks	new tpv{205,104,102}/configs/*.toml (scalar); gold/SCEC overlays; meshes, jobs/tpv*_spatial/*.sbatch, review/sign tests; planar-basis test (D1)	wire+run TPV205/102/104 (scalar); gold/SCEC overlays; np>1 MPI parity
9 Matrix Riemann (PORT)	port dynamic/godunov_flux_bimaterial_low{hpp,cpp}; pool_dispatch + modify dynamic/wave_operator.{hpp,inc} scalar path byte-unchanged; Makefile np>1 parity	BimaterialFlux ExchangeBiMaterialNeighbours_ np>1 parity
10 TPV31 (PORT)	port DepthProfile1D into heterogeneous_material.*; copy tpv31/ (config, mesh, benchmark, tests)	depth-profile matrix Riemann; LSW + depth-linear cohesion; InstantaneousOverstressCircular nucleation; SCEC eqdyna/seisol overlay
— REUSE (no change)	elasticity_operator_setup.inl already exist; orthogonal or fault_face_flux.*, ADER core, friction/* kernels, spatial_friction.cpp RS parse/resolve, InitializeFaultDOFs_Spatial_RS	byte-identical to native

9 Phases

9.1 Phase 1 – RS primitives » (aging-iterator hooks + equilibrium- ψ seed + build)

Goal

The aging RS sub-step iterator (Tpv102SubStepIterator) can be driven by a SAFS Gaussian nucleation **callback** and carries the diag hook; a per-DOF **equilibrium- ψ** seed exists; the build links the tpv102 iterator into the spatial driver. **Additive and byte-safe — no existing behaviour changes.** This phase makes RS *runnable* without touching the driver’s control flow.

Files to Modify

- dynamic/tpv102_substep_iterator.{hpp,cpp} — add a nuc_callback overload of AdvanceWithSubStepStat and a SetDiagNumLocalFaultQPs(int) member.
- dynamic/spatial_setup.hpp — add SeedEquilibriumPsi_RS(...); **add #include "../friction/dieterich_ruina.hpp"** (R-001 — the header currently includes only fault_face_flux.hpp, heterogeneous_material.hpp, spatial_friction.hpp, so DieterichRuinaFriction is otherwise undeclared). FrictionSolver (for the R-009 V0 guard) is visible transitively via fault_face_flux.hpp → friction_solver.hpp; if a future refactor drops that edge, add

#include "friction_solver.hpp" explicitly.

- miniapps/seas/Makefile — link \$(TPV102_SUBSTEP_ITERATOR_OBJ) into seas_spatial_dyn_driver; add the two new test targets.

Files to Create

- tests/unit/test_tpv102_nuc_callback_parity.cpp
- tests/unit/test_seed_equilibrium_psi_rs.cpp

Detailed Requirements

1. **Nucleation callback overload** on Tpv102SubStepIterator (declared right after the existing plain AdvanceWithSubStepStates(...) at tpv102_substep_iterator.hpp:117):

```
void AdvanceWithSubStepStates(
    std::vector<DOFData>&                dof_data,
    const std::vector<mfem::Vector>&      fault_coords,
    const std::vector<std::vector<real_t>>& Q_plus_sub,
    const std::vector<std::vector<real_t>>& Q_minus_sub,
    real_t dt_macro, real_t t_macro_start,
    real_t* I_imp_plus, real_t* I_imp_minus,
    FrictionSolver::Method method,
    const std::function<void(real_t /*t_sub_end*/,
                             real_t /*dt_sub*/)>& nuc_callback);
```

Implementation: copy the existing AdvanceWithSubStepStates body verbatim (.cpp:333–371 region) but **replace the hard-coded ApplyNucleationIncremental_TPV102(dof_data, fault_coords, t_sub_end, dt_sub) call (.cpp:326) with nuc_callback(t_sub_end, dt_sub)** at the identical point (once per sub-step, before the per-QP loop). Everything else (the ComputeStageState→slip→UpdateStateAnalytic→BuildImposedState sequence) is unchanged. Use method = FrictionSolver::Method::Brent from the caller (CLAUDE.md: Brent, not Newton). The plain overload stays for the standalone TPV102 driver.

2. **SetDiagNumLocalFaultQPs(int n)** on Tpv102SubStepIterator: store into a new member int diag_num_local_fault_qps_ = -1; (mirror Tpv205SubStepIterator:146). It is a no-op accessor for interface symmetry (TPV102 has no [SLIP] trace); having it lets the Phase-2 adapter satisfy IFrictionIterator uniformly.
3. **SeedEquilibriumPsi_RS** in dynamic/spatial_setup.hpp (**R-001 corrections inlined**):

```
inline void SeedEquilibriumPsi_RS(
    std::vector<DOFData>&                dof_data,
    const spatial::RateStatePerDOFParams& rs,    // a,Dc,V_init per-DOF
    const spatial::RateStateBlock&       blk)    // b,V_0,f_0 scalar globals
{
    // R-009: V_0 has two consumers -- the config (this seed + AgingLawPsi)
    // and the hardcoded FrictionSolver::V0 used by the force solve. They
    // MUST agree or the fault is not in equilibrium at t=0. (Proper fix:
```



```

// thread blk.V_0_default into FrictionSolver; minimal guard for now.)
MFEM_VERIFY(std::abs(blk.V_0_default - FrictionSolver::V0)
    <= 1e-30 + 1e-12 * FrictionSolver::V0,
    "SeedEquilibriumPsi_RS: [friction.rate_state].V_0 ("
    << blk.V_0_default << ") must equal FrictionSolver::V0 ("
    << FrictionSolver::V0 << "); the force solve hardcodes V0.");
for (std::size_t i = 0; i < dof_data.size(); ++i) {
    DOFData& d = dof_data[i];
    // R-001 edge guard: else log()/InitialStatePsi -> inf/NaN psi.
    MFEM_VERIFY(rs.V_init(i) > 0.0,
        "SeedEquilibriumPsi_RS: V_init <= 0 at DOF " << i
        << " (ill-posed steady state)");
    const DieterichRuinaFriction fr(
        DieterichRuinaFriction::Constants{
            blk.V_0_default, blk.f_0_default, blk.b_default, rs.Dc(i)});
    const real_t tau0 = std::hypot(d.tau1_0 + d.tau1_nuc,
        d.tau2_0 + d.tau2_nuc);
    // R-001: damping is d.eta_s (== rs.eta(i) == 0.5*sqrt(mu*rho) for
    // homogeneous SAFS, and exactly what FaultFaceFlux uses in the
    // solve). NEVER rs.eta_s -- RateStatePerDOFParams has no such member.
    d.psi = (tau0 > 0.0)
        ? fr.InitialStatePsi(tau0, rs.V_init(i),
            d.sigma_n0, d.eta_s, rs.a(i))
        : (blk.f_0_default
            + blk.b_default * std::log(blk.V_0_default / rs.V_init(i)));
}
}

```

This overwrites the `d.psi = 0.0` stub at `spatial_setup.hpp:341`. The `tau0==0` branch uses the locked steady-state ψ ($f_0 + b \cdot \ln(V_0/V_{\text{init}})$). **R-001 (CRITICAL)**: the radiation-damping argument is `d.eta_s` (a `DOFData` member, set at `spatial_setup.hpp:82`), **not** `rs.eta_s` — `RateStatePerDOFParams` has only a per-DOF `eta` Vector (`spatial_friction.hpp:295`); `rs.eta_s` is a hard compile error. For the homogeneous SAFS material `d.eta_s == rs.eta(i) == 0.5*sqrt( $\mu\rho$ )` at the same element centroid, so the seed's damping matches the friction solve's. (Confirm `DieterichRuinaFriction::Constants{V0,f0,b, Dc}` field order and `InitialStatePsi(tau0,V_init,sigma_n,eta,a)` at impl time — both verified at `friction/dieterich_ruina.hpp:43-49,506-507`.)

R-009 (V0 single source of truth): the leading `MFEM_VERIFY` asserts the config `blk.V_0_default` equals the solver's compile-time `FrictionSolver::V0` ($= 1e-6$, `friction_solver.hpp:60`), which the force solve hardcodes (`fault_face_flux.cpp:224`: $C = \exp(\psi/a)/(2 \cdot \text{FrictionSolver::V0})$). Without it, a config that sets `[friction.rate_state].V_0  $\neq 1e-6$` seeds ψ (and runs `AgingLawPsi`) against one `V_0` while the solve uses another $\rightarrow$ the fault is **not** at `V_init` at $t=0$ and the friction is wrong throughout, with no error (the `RateStateBlock` default is $1e-6$, so a default config is only *accidentally* consistent). `f_0/b` are immune (the regularized $f =$

$a \cdot \text{asinh}[V/(2V_0) \cdot \exp(\psi/a)]$ is single-sourced from `blk` in both seed and ψ -update); only `V_0` has the dual-source hazard. The minimal guard above is byte-safe for the default-`V_0` deliverable; the **proper** fix (a follow-up, per the no-hardcoded-numbers rule) threads the resolved `V_0` into `FrictionSolver/CompleteFromVabs`, removing the `constexpr` entirely — but that touches the byte-exact TPV oracle path, so it is gated out of the priority run.

Edge Cases

- `tau0 == 0` (no pre-stress shear) → seed the locked steady state, not `InitialStatePsi` (which divides through the imposed traction).
- `rs.V_init(i) <= 0` → abort in `SeedEquilibriumPsi_RS` (ill-posed steady state).
- **R-009**: `blk.V_0_default != FrictionSolver::V0` → abort (dual-source `V_0`); the deliverable uses the default `V_0 = 1e-6`, so the guard passes and is a tripwire for future tuning.
- The plain `AdvanceWithSubStepStates` overload must remain byte-identical for the standalone TPV102 driver (the new overload is strictly additive).

Acceptance Criteria

- `seas_test_tpv102_nuc_callback_parity` (**R-008 — pin one method**): on a 2-QP, `O=2` fixture, (a) the new overload called with `FrictionSolver::Method::NewtonRaphsonStable` (the *same* method the plain overload defaults to, `tpv102_substep_iterator.hpp:127`) and `nuc=&](t,dt){ApplyNucleationIncremental_TPV102(dof,coords,t,dt);}` reproduces the plain overload **bit-for-bit** (`EXPECT_EQ` on `psi,slip1/2,V1/2,tau*_corr,sigma_n_corr,I_imp_±`). The parity fixture **must not** use Brent on one side and the default on the other (a legitimate solver-difference would masquerade as a regression). (b) The new overload with an **empty** callback matches a reference run with nucleation suppressed.
- A **second test case in the same target** (`test_tpv102_nuc_callback_parity.cpp`, **R-008 — non-parity Brent**): the new overload with `FrictionSolver::Method::Brent` (the production RS method, `CLAUDE.md`) produces **finite**, **sign-consistent** `tau*_corr` and **monotone** ψ over the sub-steps — a smoke that Brent is wired, **not** a bit-for-bit equality (Brent $\neq$ `NewtonRaphsonStable`, so `EXPECT_EQ` would be wrong here).
- `seas_test_seed_equilibrium_psi_rs` (**R-001 — extends the seed test**): for a synthetic DOF (`tau0=29.2e6`, `σ_n0=49.27e6`, `V_init=1e-9`, `a=0.010`, `Dc=2.0`, `eta=0.5·sqrt(μp)`, global `b=0.015`, `V0=1e-6`, `f0=0.6`), after `SeedEquilibriumPsi_RS` the steady residual $|\text{tau0} - (\sigma_{n0} \cdot a \cdot \text{asinh}[V_{\text{init}}/(2V_0) \cdot \exp(\psi/a)] + \text{eta} \cdot V_{\text{init}})| < 1e-6 \cdot \text{tau0}$. This guards **both** the member name (`d.eta_s`, not `rs.eta_s`) **and** the seed↔solve η consistency. A second case with `tau0=0` asserts $\psi == f_0 + b \cdot \ln(V_0/V_{\text{init}})$ (locked branch), and a `V_init=0` case asserts the new `MFEM_VERIFY` aborts.
- **R-009**: in the same target, a `blk.V_0_default = 2e-6` ($\neq$ `FrictionSolver::V0 = 1e-6`) case asserts `SeedEquilibriumPsi_RS` **aborts** on the `V_0` guard; the control `blk.V_0_default = 1e-6` passes and (optionally) verifies a `t=0` solve sits at `V_init` ($|V - V_{\text{init}}| \leq 1e-6 \cdot V_{\text{init}}$) while `2e-6` *without* the guard would be $O(1)$ off (documents the silent-wrong behaviour the guard prevents).
- `make -n seas_spatial_dyn_driver` shows $\$(\text{TPV102_SUBSTEP_ITERATOR_OBJ})$ on the link line; all three new/extended test targets resolve.

Dependencies

Depends on: nothing. Required by: Phases 2, 3.

9.2 Phase 2 – Friction strategy interface » (IFrictionIterator + adapters + factory)

Goal

A minimal IFrictionIterator strategy interface with two adapters over the **existing, proven** iterators (LSW→Tpv205SubStepIterator, aging→Tpv102SubStepIterator) and a MakeFrictionIterator factory; AdvanceADERWithSubStep_Spatial takes IFrictionIterator&. **The existing LSW path is byte-unchanged** (the adapter is a transparent forwarder). This is the runtime-dispatch foundation; the full method-oriented unification is Phase 5.

Files to Create

- dynamic/friction_iterator.hpp — IFrictionIterator + LswFrictionIterator + RateStateAgingFrictionIterator
- dynamic/friction_iterator_factory.{hpp,cpp} — MakeFrictionIterator.
- tests/unit/test_friction_iterator_factory.cpp.
- tests/unit/test_advance_interface_compiles.cpp (**R-010** — compile-only: the retyped AdvanceADERWithSubStep_Spatial signature + a stub IFrictionIterator whose only sub-step entry is Advance(...) must compile).

Files to Modify

- drivers/spatial_dyn_driver.cpp — change AdvanceADERWithSubStep_Spatial's iterator parameter (:367) from Tpv205SubStepIterator& to IFrictionIterator&; **R-010**: rename the body's iterator.AdvanceWithSubStepStates(...) call at :439 to iterator.Advance(...) (the only call that changes; GetDeltaT/GetTimeWeights/SetSubSteps are unchanged). Replace the concrete Tpv205SubStepIterator substep_iterator(fault_flux); construction (:1736) with auto fr = std::make_unique<LswFrictionIterator>(fault_flux); and pass *fr. (The RS branch is Phase 3; here only LSW is wired, so behaviour is identical.)

Detailed Requirements

1. **IFrictionIterator** (minimal; SRW arrives in Phase 5):

```
class IFrictionIterator {
public:
    virtual void SetSubSteps(std::vector<real_t> dT,
                           std::vector<real_t> w) = 0;
    // R-010: AdvanceADERWithSubStep_Spatial's body also reads these two
    // (driver :384/:385/:404), so they MUST be on the interface or the
    // parameter retype (req 5) will not compile.
    virtual const std::vector<real_t>& GetDeltaT() const = 0;
    virtual const std::vector<real_t>& GetTimeWeights() const = 0;
    virtual void Advance(
        std::vector<DOFData>& dof, const std::vector<mfem::Vector>& coords,
        const std::vector<std::vector<real_t>>& Qp,
        const std::vector<std::vector<real_t>>& Qm,
```

```

    real_t dt, real_t t0, real_t* Iimp_p, real_t* Iimp_m,
    const std::function<void(real_t,real_t)>& nuc_cb) = 0;
virtual void SetDiagNumLocalFaultQPs(int n) = 0;
virtual FaultFrictionLaw WaveOpLaw() const = 0;
virtual ~IFrictionIterator() = default;
};

```

R-010 note: the reviewer flagged only the :439 call rename, believing GetDeltaT/GetTimeWeights were already on the interface — they were **not** in the rev-5 draft. Both Tpv205SubStepIterator and Tpv102SubStepIterator already expose them (tpv205...hpp:138–139, tpv102...hpp:130–133), so the adapters forward trivially; without them on the interface, the retype breaks at :384/:385/:404 too. They are added above.

2. **LswFrictionIterator** owns a Tpv205SubStepIterator `it_`; (ctor takes FaultFaceFlux&). `Advance(...)` forwards to `it_.AdvanceWithSubStepStates(dof, coords, Qp, Qm, dt, t0, Iimp_p, Iimp_m, nuc_cb)` (Tpv205's existing callback overload). `WaveOpLaw()` → `FaultFrictionLaw::LSW.SetSubSteps/GetDeltaT/GetTimeWeights/SetDiagNumLocalFaultQPs` forward to `it_`.
3. **RateStateAgingFrictionIterator** — **member order matters** (the iterator holds a `const AgingLawPsi&`, so the law must be declared/constructed first):

```

class RateStateAgingFrictionIterator : public IFrictionIterator {
public:
    RateStateAgingFrictionIterator(FaultFaceFlux& flux,
                                   const spatial::RateStateBlock& blk)
        : law_(blk.b_default, blk.V_0_default, blk.f_0_default),
          it_(flux, law_) {}
    void Advance(/*...*/,
                 const std::function<void(real_t,real_t)>& nuc_cb) override
    { it_.AdvanceWithSubStepStates(
        /*...*/, FrictionSolver::Method::Brent, nuc_cb); }
    FaultFrictionLaw WaveOpLaw() const override
    { return FaultFrictionLaw::RateAndState; }
    /* SetSubSteps / GetDeltaT / GetTimeWeights / SetDiagNumLocalFaultQPs
       forward to it_ */
private:
    AgingLawPsi law_; // declared BEFORE it_
    Tpv102SubStepIterator it_;
};

```

It uses the Phase-1 `nuc_callback` overload and `FrictionSolver::Method::Brent`. **R-009:** `law_(blk.b_default, blk.V_0_default, blk.f_0_default)` makes `AgingLawPsi` read the config `V_0`, while the force solve hardcodes `FrictionSolver::V0` — the Phase-1 `SeedEquilibriumPsi_RS` guard (and, defensively, `MakeFrictionIterator`) asserts they agree.

4. **MakeFrictionIterator:**

```
std::unique_ptr<IFrictionIterator> MakeFrictionIterator(
    const spatial::SpatialFrictionConfig& cfg, FaultFaceFlux& flux,
    const spatial::RateStatePerDOFParams* rs /*nullptr for LSW*/);
```

cfg.law==SlipWeakening → make_unique<LswFrictionIterator>(flux); cfg.law==RateState → require cfg.rate_state.has_value(), then make_unique<RateStateAgingFrictionIterator>(flux, *cfg.rate_state). **SRW guard:** if a future state_evolution==slip_law_srw field is set, abort with "slip-law SRW lands in Phase 5" (aging is the only RS law wired in Phases 1–3). **R-009 defensive guard:** on the RateState branch, also MFEM_VERIFY(|cfg.rate_state->V_0_default - FrictionSolver::V0| ≤ tol, ...) (same invariant as the Phase-1 seed guard — cheap, and catches a mis-set V_0 before the iterator is built).

5. **AdvanceADERWithSubStep_Spatial** parameter retype: IFrictionIterator& iterator (was Tpv205SubStepIterator&, :367). **R-010 (CRITICAL for build):** the body does **not** already call iterator.Advance(...) — at :439 it calls iterator.AdvanceWithSubStepStates(dof_data, fault_coords, Q_pointwise_plus, Q_pointwise_minus, dt_step, t_step_start, I_imp_plus_flat.data(), I_imp_minus_flat.data(), nuc_callback). **Rename that call to iterator.Advance(...)** with the identical argument list (the interface names the entry point Advance). The body's other iterator calls — GetDeltaT() (:384), GetTimeWeights() (:385/:404), SetSubSteps() (:403) — are now all on the interface (req 1) and need no change. No other logic change; all safes fixes stay.

Edge Cases

- Member-init order in RateStateAgingFrictionIterator (law_ before it_) — a reversed order dangles the const AgingLawPsi&; enforce via the declared order above.
- SRW requested in Phases 1–3 → clean "Phase 5" abort, not a silent fallback.
- **R-010:** the interface must declare **every** method the driver body invokes on iterator (SetSubSteps, GetDeltaT, GetTimeWeights, Advance, SetDiagNumLocalFaultQPs) — a missing one is a compile error at the retype, not a runtime issue.

Acceptance Criteria

- ☐ seas_test_friction_iterator_factory: SlipWeakening → LswFrictionIterator (WaveOpLaw()==LSW); RateState(aging) → RateStateAgingFrictionIterator (WaveOpLaw()==RateAndState); SRW → abort.
- ☐ **R-010:** seas_spatial_dyn_driver **compiles** after the retype — i.e. the :439 call is Advance(...) and the interface declares GetDeltaT/GetTimeWeights. test_advance_interface_compiles is the minimal compile-only guard (pre-fix: fails because the call site uses AdvanceWithSubStepStates, absent from the interface).
- ☐ **SAFS LSW no-regression:** an existing SAFS LSW config runs **byte-identical** to a pre-Phase-2 checkpoint (the adapter forwards to the same Tpv205SubStepIterator).
- ☐ seas_spatial_dyn_driver builds and the LSW smoke runs unchanged.

Dependencies

Depends on: Phase 1. Required by: Phase 3.

9.3 Phase 3 – SAFS + rate-and-state run » (driver RS branch + D3.1 + config) – THE NEAR-FUTURE GOAL

Goal

spatial_dyn_driver runs the **SAFS Dc2 problem with aging rate-and-state friction** from a TOML; an 8-rank smoke produces a physical RS rupture. Also lands the D3.1 right-lateral-positive constant_tensor convention (bit-identical on-fault). **This phase delivers the near-future goal.**

Files to Modify

- drivers/spatial_dyn_driver.cpp — remove the is_lsw reject; add the RS resolver/DOF-init branch (**R-002**: gate the :1146 slip_weakening.has_value() guard under is_lsw, outer-scope lsw+rs); D3.1 factory negation at the ConstantTensorStressSource construction (:1123–1135); SetFaultFrictionLaw from is_lsw; **R-005**: branch the ParaView “state” channel (:1809) so RS writes d.psi.
- spatial/code/spatial_friction.cpp — **R-006**: in resolve_rs_impl’s per-rule loop (:1014/1017/1018), **reject** per-DOF b/V_0/f_0 spatial overrides (the aging law and DOFData are scalar in those), so the plan’s “rejected by ResolveRateState” claim holds. **R-011**: relax the MFEM_VERIFY(a_i < b_i) validator at :1070 to **allow** a > b (velocity-strengthening) — keep only a>0, b>0, finite. VS regions are how aging-law ruptures arrest at the fault edges (TPV102 itself uses a high-a border).
- safs/project_7.0_alternative/config/spatial_friction_slip_weakening_safs_projected_stress*.toml (Dc2/Dc8/Dc10) — flip sigma_xy_pa −9.888...e6 → +9.888...e6 **together with** the D3.1 negation, **in one atomic commit** with the factory change (**R-007**; net on-fault unchanged).

Files to Create

- safs/project_7.0_alternative/config/spatial_friction_rate_state_safs_projected_stress_resolution.toml
- jobs/safs/spatial_dyn_resolution_Dc2_ratestate_8N_400r_dev_2hr_safs.sbatch
- tests/unit/test_constant_tensor_sign.cpp (**R-007** — D3.1 golden bit-identity, **parametrized over Dc2/Dc8/Dc10 + the RS config**)
- tests/unit/test_resolve_rate_state_guards.cpp (**R-003** no-double-PP + **R-006** per-DOF b/V_0/f_0 rejection + **R-011** a > b velocity-strengthening allowed)
- tests/unit/test_paraview_state_channel_rs.cpp (**R-005** — RS state channel writes ψ)

Detailed Requirements

1. **Remove the reject** at spatial_dyn_driver.cpp:716–719 (MFEM_VERIFY(is_lsw, "...rate_state path is a deferred follow-up...")). Keep the `cfg.law ∈ {SlipWeakening, RateState}` verify (:710–714) and is_lsw (:715).
2. **Friction-law tag** at :971: replace the hard-coded `wave.SetFaultFrictionLaw(FaultFrictionLaw::LSW)` with `‘wave.SetFaultFrictionLaw(is_lsw ? FaultFrictionLaw::LSW : FaultFrictionLaw::RateAndState);(is_lsw‘ already in scope)`.
3. **Resolver branch (R-002 + R-003)** — replaces driver :1146–1152. The **un-gated** MFEM_VERIFY(cfg.slip_weakening) at :1146 **must move inside if (is_lsw)**: the parser forbids a [friction.slip_weakening] block when law="rate_state" (spatial_friction.cpp:809–814), so an RS config has `cfg.slip_weakening == nullopt` and the literal :1146 guard would **abort the SAFS-RS run**

during setup (wrong message, before nucleation/DOF-init). Both result structs are declared at **outer scope** so the DOF-init at :1220 (req 4) still sees them:

```

spatial::SpatialFrictionResolver    resolver;
spatial::SlipWeakeningPerDOFParams  lsw; // outer scope; filled iff is_lsw
spatial::RateStatePerDOFParams      rs;  // outer scope; filled iff !is_lsw
if (is_lsw) {
    MFEM_VERIFY(cfg.slip_weakening.has_value(), // R-002: now gated
                "law=slip_weakening requires [friction.slip_weakening]");
    lsw = resolver.ResolveSlipWeakening(*cfg.slip_weakening,
                                        dof_coords_3d, dof_to_attr);
} else {
    MFEM_VERIFY(cfg.rate_state.has_value(),
                "law=rate_state requires [friction.rate_state]");
    rs = resolver.ResolveRateState(
        *cfg.rate_state, dof_coords_3d, dof_to_elem, dof_to_attr,
        material, pmesh,
        spatial::PorePressureSpec{}, // R-003: NOT cfg.stress.pore_pressure
        geom.sigma_n_per_dof());    // sigma_n_per_dof() is ALREADY effective
} // input already effective; non-zero pp would double-subtract P_p

```

R-003 (double pore-pressure): ResolveRateState’s last argument is named `sigma_n_total_per_dof` and it subtracts `pp` internally (`spatial_friction.cpp:1024–1039`) to fill `rs.sigma_n_eff`. But `geom.sigma_n_per_dof()` is **already** effective — it comes from `ProjectFaultPreStress`, “the standard effective normal stress $\sigma_n - P_p$ ” (`field_coefficient.hpp:231`). Passing `cfg.stress.pore_pressure` (Dc2: $P_p=16$ MPa) here would give `rs.sigma_n_eff` = total – $2 \cdot P_p$ (≈ 33 MPa vs the correct ≈ 49 MPa). It is *latent* today (no consumer reads `rs.sigma_n_eff`; physics uses `d.sigma_n0`), but a real trap and it silences the internal `sigma_n_eff > 0` validator — so pass a **zero** `PorePressureSpec{}` and document it at the call site. (verbatim signature, `spatial_friction.hpp:306–326`.)

4. **DOFData init branch:** `is_lsw` → existing IP-aware `InitializeFaultDOFs_Spatial<ParMesh>(...)` (:1220–1224); else

```

InitializeFaultDOFs_Spatial_RS<ParMesh>(
    dof_data, ndof, dof_to_elem, material, pmesh, rs,
    geom.GetTauPre(), geom.sigma_n_per_dof()); // sig: spatial_setup.hpp:301
SeedEquilibriumPsi_RS(dof_data, rs, *cfg.rate_state); // Phase-1 helper

```

Note: `InitializeFaultDOFs_Spatial_RS` has **no IP-aware overload** (uses element centroid). SAFS material is **homogeneous**, so centroid $\equiv$ IP — correct for SAFS; flag an IP-aware RS overload as a heterogeneous-material follow-up (needed only if RS meets a spatially-varying material, i.e. not in this plan).

5. **Iterator + nucleation** (replace the Phase-2 LSW-only construction at :1736): `auto fr = MakeFrictionIterator(cfg, fault_flux, is_lsw ? nullptr : &rs); fr->SetSubSteps(deltaT, weights); fr->SetDiagNumLocalFaultQPs(wave.GetNumLocalFaultQPs()); Nucleation is`

reused unchanged — the existing ResolveGradualOverstress (:1162–1167) and the nuc_cb lambda (:1753–1761) are law-agnostic and flow through AdvanceADERWithSubStep_Spatial→fr→Advance→aging adapter→the Phase-1 Tpv102 overload.

6. **D3.1 factory negation** at the ConstantTensorStressSource construction (:1123–1135):

```
spatial::ConstantTensorStressSource src(
    cfg.stress.sigma_xx_pa, cfg.stress.sigma_yy_pa, cfg.stress.sigma_zz_pa,
    -cfg.stress.sigma_xy_pa,           // D3.1: right-lateral-+ input
    +cfg.stress.sigma_yz_pa, +cfg.stress.sigma_xz_pa);
```

with a documented comment block (normals compression-positive; strike shear right-lateral-positive; $\tau_{\text{strike}}=+\sigma_{xy_pa}$). **R-007 (atomicity)**: the factory negation and the σ_{xy_pa} flip in **all** affected configs (Dc2/Dc8/Dc10 **and** the new RS config) must land in **one commit** — a half-applied change (factory flipped but a config not, or vice-versa) silently inverts the on-fault shear *direction* on the un-flipped config. The parametrized golden test (req 8) guards every flipped config so a partial application fails loudly. *(D3.1 is net-zero physics — kept in Phase 3 per the user’s standing decision; the review’s alternative was to defer it to Phase 4, but the atomic-commit + all-config golden test fully closes the partial-application hazard, so it stays.)*

7. **SAFS-RS config** — clone the LSW Dc2 config; set [meta].law = "rate_state"; add [friction.rate_state] (a_default, b_default, Dc_default, V_0_default, f_0_default, V_init_default, sigma_n_default, eta="auto" — all already parsed, spatial_friction.cpp:388–466); keep [stress] constant_tensor with the post-D3.1 positive σ_{xy_pa} ; keep [nucleation] gradual_overstress (Gaussian). Do **not** set interior_flux (the driver defaults to the scalar WaveOperator ctor; the matrix option is Phase 9). Per-DOF a/Dc [...spatial]] rules are allowed; per-DOF b/V_0/f_0 are **rejected by ResolveRateState** (the aging law + DOFData are scalar in those — see req 11, **R-006**: the rejection does not exist today and is added in this phase). **R-009**: keep V_0_default = 1e-6 (= FrictionSolver::V0); any other value trips the Phase-1 seed guard. **R-011**: the first SAFS+RS smoke is **fully velocity-weakening** (a < b everywhere → a_default = 0.010, b_default = 0.015), which the existing validator permits and which runs to the mesh boundary (does not arrest); a TPV102-faithful **bounded** rupture adds a velocity-strengthening (a > b) border, which requires the req-12 validator relaxation.
8. test_constant_tensor_sign.cpp (D3.1, **R-007 — parametrized over {Dc2,Dc8,Dc10, rate_state}**): for **each** flipped config, build its stress block via the driver’s factory construction path (so the negation is exercised), project onto a canonical-frame DOF, assert $\tau_{\text{strike}}=+\sigma_{xy_pa}$ (right-lateral positive), $\sigma_n=+\sigma_{yy_pa}$, $\tau_{\text{dip}}=+\sigma_{yz_pa}$, **bit-identical to the pre-change golden literals** captured from each current config (EXPECT_NEAR(...,0.0)); precondition dip==(0,0,-1), strike==(+1,0,0). This is what makes the multi-config flip safe — every flipped config is golden-guarded, not just Dc2.
9. sbatch sibling of the LSW Dc2 job (8N/400r dev, 2 h) with SAFS_CONFIG → the new RS config; keep the diagnostic env toggles.
10. **ParaView RS state channel (R-005)** at spatial_dyn_driver.cpp:1809: the snapshot writer

hard-codes the “state” field as an LSW friction coefficient `LSWFrictionCoefficient_TPV205(delta_norm, d.lsw_mu_s, d.lsw_mu_d, d.lsw_d_c)`. `InitializeFaultDOFs_Spatial_RS` never sets `lsw_mu_s/mu_d/d_c`, so for RS they are 0 and the channel is 0 (or NaN at `delta==0`, step 0) — the very field that should let the user watch the RS state is garbage. Branch it (`is_lsw` is in scope):

```
if (is_lsw) {
    pv_local_state(i) = mfem::seas::LSWFrictionCoefficient_TPV205(
        delta_norm, d.lsw_mu_s, d.lsw_mu_d, d.lsw_d_c);
} else {
    pv_local_state(i) = d.psi;    // RS state var (matches tau*_corr solve)
}
```

Writing ψ is the cheapest correct choice (the RS friction coefficient is redundant with `slip_rate/tau*_corr`). The LSW branch is byte-identical to today.

11. **Reject per-DOF `b/V_0/f_0` in `ResolveRateState` (R-006)** — `spatial/code/spatial_friction.cpp`, in `resolve_rs_impl`’s per-rule loop (where it currently *applies* them at `:1014/1017/1018`). The aging law is built from the **global** `blk.b_default/V_0_default/f_0_default` (Phase-2 ctor) and `DOFData` has no per-DOF `b/V_0/f_0` slots, so a per-DOF override is silently dropped by every physics path — a “knob that does nothing.” Add an abort:

```
MFEM_VERIFY(std::isnan(r.b) && std::isnan(r.f_0) && std::isnan(r.V_0),
    "ResolveRateState: per-DOF b/f_0/V_0 spatial overrides are not "
    "supported (AgingLawPsi is scalar); set them only in the "
    "[friction.rate_state] defaults. Offending rule at DOF " << i);
```

(Place it before the `if (!std::isnan(r.b))...` assignments, or drop those assignment lines and the unused `rs.b/f_0/V_0` members entirely after a grep confirms no consumer — the abort is the minimal guard for the deliverable. Per-DOF `a/Dc/V_init/sigma_n/eta` stay allowed.)

12. **Relax the `a < b` validator (R-011)** — `spatial/code/spatial_friction.cpp:1070`. Replace `MFEM_VERIFY(a_i < b_i, "ResolveRateState: a >= b at DOF " << i)` so that velocity-strengthening (`a > b`) is **allowed**; keep the positivity/finiteness checks (`a_i > 0`, `b_i > 0`):

```
MFEM_VERIFY(a_i > 0.0, "ResolveRateState: a <= 0 at DOF " << i);
MFEM_VERIFY(b_i > 0.0, "ResolveRateState: b <= 0 at DOF " << i);
// R-011: a > b (velocity-strengthening) is allowed -- it is how aging-law
// ruptures arrest at the fault edges (TPV102 uses a high-a border). Only
// require a, b finite and > 0; do NOT require a < b.
```

Scope: the first deliverable smoke is fully VW (req 7), so this is the **only** round-2 finding strictly *optional* for the immediate run. It is included in Phase 3 because it is a 2-line change that directly enables the user’s “tpv102-like” (bounded) target and removes a setup-abort trap; landing it now avoids a second touch of `ResolveRateState`. Do **not** remove the positivity checks. (*Pre-existing validator — not introduced by this plan; relaxing it is the documented prerequisite for any bounded SAFS-RS rupture.*)

Edge Cases / Risk gate

- **Shared-fault RS ψ is 1st-order in time at $\text{ader_order} \geq 2$ (top risk; review R-004).** For shared (rank-seam) fault QPs the driver runs **inline** EvaluateADER at macro dt (`wave_operator.inl:4908`), *not* the per-sub-step iterator (the R-1601 fallback). The iterator advances `d.psi` for **all** QPs incl. shared (`tpv102...cpp:348`), then the inline shared flux reads that **end-of-step** ψ for the whole macro step and EvaluateADER does **not** mutate ψ (`assert fault_face_flux.cpp:422`). Net: interior QPs get a sub-step-consistent ψ trajectory; shared QPs use end-of-step $\psi \rightarrow$ **1st-order on shared faces** (SAFS default $\text{ader_order}=2$). This is **bounded, not a blow-up**: it is structurally the *same* as the **working** LSW slip path (the iterator owns slip; inline EvaluateADER_LSW reads end-of-step slip and does not re-accumulate, `fault_face_flux.cpp:815–824`). The reconcile **already broadcasts `d.psi`** (`payload[5], :5304/:5327`), so cross-rank ψ is single-valued — the issue is **temporal accuracy, not cross-rank desync and not a double-advance**. Added risk over LSW: ψ enters friction nonlinearly ($\exp(\psi/a)$), so the end-of-step approximation is less benign than slip $\rightarrow \mu_{\text{eff}}$. **Gate (R-004):** keep RS $n_p > 1$ “open until validated”; the consistency check is now **two-pronged** — (a) cross-rank `d.psi` bit-identity post-reconcile, **and (b) a temporal interior-vs-shared ψ drift** at the same physical QP within tolerance (see the $n_p=2$ test). If (b) exceeds tolerance, the documented follow-up is to drive shared QPs through a sub-step ψ path. **Do not present 8-rank SAFS-RS results as validated until (a)+(b) pass.**
- Equilibrium ψ at $\tau_{00}=0 \rightarrow$ locked steady state (Phase-1 helper handles it).
- D3.1 half-applied (factory negation XOR a config flip) $\rightarrow$ the parametrized `test_constant_tensor_sign` (req 8) golden mismatch fails loudly on the un-flipped config.

Acceptance Criteria

- **R-002:** the new RS SAFS config (no `[friction.slip_weakening]` block) **does not abort** at the `slip_weakening.has_value()` guard. `test_rate_state_config_no_abort`: run the driver setup path to just past DOF-init in `--dry-run`; assert it reaches the law: `rate_state` banner and constructs `dof_data` (pre-fix: aborts at `:1146`).
- SAFS-RS Dc2 `--dry-run --verify-dispatch` reports `RateAndState` + the aging iterator + Gaussian nucleation.
- **R-003:** `test_resolve_rate_state_no_double_pp` (in `test_resolve_rate_state_guards.cpp`): $N=1$ DOF, `sigma_n_total_per_dof={49.27e6}` (already effective), `pp=PorePressureSpec{}` $\rightarrow$ `rs.sigma_n_eff(0)==49.27e6` (1 ULP); control with `pp.P_p_pa=16e6` yields `33.27e6` (documents the trap).
- **R-006:** `test_resolve_rate_state_rejects_per_dof_b` (same file): a `RateStateBlock` with one spatial rule `{kind=Depth, b=0.02}` $\rightarrow$ `ResolveRateState` aborts with the “per-DOF `b/f_0/V_0` not supported” message; control rule setting only `a/Dc` resolves cleanly.
- **R-011:** `test_resolve_rate_state_allows_velocity_strengthening` (same file): one DOF with `a=0.02, b=0.015` ($a > b$) $\rightarrow$ `ResolveRateState` does **not** abort; control `a<=0` still aborts. Pre-fix: the $a < b$ MFEM_VERIFY rejects the $a > b$ case.
- **R-005:** `test_paraview_state_channel_rs`: after one RS macro-step, `pv_local_state(i)==dof_data[i].psi` for all `i` and finite at step 0 (`delta==0`); the LSW branch is byte-identical to the pre-change writer.
- **R-007:** `seas_test_constant_tensor_sign` passes for **all** of `{Dc2,Dc8,Dc10, rate_state}`;

an existing SAFS **LSW** run is byte-identical after the `sigma_xy_pa` flip (D3.1 net-zero).

- **R-004:** `test_shared_fault_rs_psi_consistency_np2` (MPI `np=2`, one shared fault QP): after one macro-step (`ader_order=2`), (a) `(*fault_dof_data_)[shared].psi` is bit-identical on both ranks post-reconcile; (b) $|\psi_{\text{shared}} - \psi_{\text{interior_twin}}| \leq \text{tol}$ (`twin` = same physical QP through the interior sub-step path); record `tol` in the benchmark doc.
- » **Working SAFS+RS:** an 8-rank short smoke runs — pre-nucleation the fault sits at $V \approx V_{\text{init}}$ (locked steady state from the ψ seed); nucleation triggers an RS rupture; `max_slip` is physical; the R-004 (a)+(b) checks pass (or the shared-fault-RS follow-up is opened and `np>1` RS is **not** presented as validated).

Dependencies

Depends on: Phases 1, 2. Required by: Phases 4–10 build on the working driver.

9.4 Phase 4 – Name generalization (D-rename)

Goal

`FaultGeometry::ComputeParams/HasParams` is the project-wide name; no behaviour changes. (Deferred behind the SAFS+RS goal because it is purely cosmetic — the RS path in Phases 1–3 uses the existing `ComputeSAFSParams` name.)

Files to Modify

- `fault/fault_geometry.hpp`, `fault/fault_geometry_safs.inl`, `fault/rate_state_fault.hpp`, `config/seas_config.hpp`, `config/seas_config_bridge.hpp`, `spatial/code/spatial_stress.{hpp,cpp}`, `drivers/spatial_dyn_driver.cpp`, `tests/unit/test_compute_safs_params.cpp`

Detailed Requirements

1. Mechanical rename `ComputeSAFSParams` → `ComputeParams`, `HasSAFSParams` → `HasParams`, `safs_params_computed_` → `params_computed_` across all files (incl. the call site `seas_config_bridge.hpp:147` and the driver `:1131–1140`). Fault kind stays informational.
2. **Do not** add the reference R-001 sign flip — the no-flip projection rule and `SEAS_ZERO_DIP_PRESTRESS` stay verbatim (nothing reverted; see §2-D3.1).

Acceptance Criteria

- `grep -rn 'ComputeSAFSParams\|HasSAFSParams' miniapps/seas` returns nothing.
- `seas_test_compute_safs_params` (incl. `T_65_5` templated==sidecar) and the SAFS+RS smoke both pass unchanged after the rename.

Dependencies

Depends on: Phase 3. Required by: nothing (independent cleanup).

9.5 Phase 5 – Unified method-oriented iterator (replaces the Phase-2 adapters)

Goal

The three benchmark-named iterators collapse into the method-oriented set (`LinearSlipWeakeningIterator`, `RateStateSubStepIterator<RateStateAgingPolicy>`, `RateStateSubStepIterator<RateStateSlipLawSrwPolicy>`) sharing one `RunSubSteps_` skeleton behind `IFrictionIterator`; the Phase-2 adapters are re-pointed to the unified classes, parity-tested bit-for-bit against the standalone oracle. Adds the SRW law.

Files to Create

- `dynamic/friction_substep_iterator.{hpp,cpp}` — `SubStepIteratorBase` (skeleton), `RateStateSubStepIterator<StatePolicy>`, `LinearSlipWeakeningIterator`, the two using aliases; explicit template instantiations of both policies in the `.cpp`.
- `friction/state_policies.hpp` — `RateStateAgingPolicy`, `RateStateSlipLawSrwPolicy` (§5.1).
- `tests/unit/test_friction_substep_iterator_parity.cpp`.

Files to Modify

- `dynamic/friction_iterator_factory.cpp` — re-point `MakeFrictionIterator` to the unified classes (the `RateStateAgingFrictionIterator`/`LswFrictionIterator` shells either wrap the unified iterator or are replaced by the unified classes implementing `IFrictionIterator` directly); extend `Advance` to the full signature and add the SRW branch.
- **None of the standalone `tpv{205,102,104}_substep_iterator.*`** (oracle, untouched).

Detailed Requirements

1. `SubStepIteratorBase::RunSubSteps_` lifts the common envelope verbatim (the `SetSubSteps` validator, `Advance*` preconditions, `memset(I_imp_*,0)`, the per-(o,i) $Q_{\pm}$ build from `I/dt_macro` or `Q_pointwise[o]` via a `QSource` adapter, `accum_scale` accumulation, slip accumulation, `WriteBackState` on the last sub-step, `nuc_fn(t_sub_end,dt_sub)` once per sub-step before the QP loop). See §5.1 for the full structure.
2. `RateStateSubStepIterator<StatePolicy>` per-QP `step_fn`: `flux_.ComputeStageState(d,Qp,Qm,s,method)` → slip accumulation → `d.psi = StatePolicy::UpdatePsi(law_, d, s.V_abs, dt_sub, extra_, i)` → `flux_.BuildImposedState(...)`. Aging policy → `UpdateStateAnalytic(...)`; SRW policy → `UpdateStateAnalyticSlipLawSRW(..., V_w[i], ...)`. The SRW `V_w` span is owned by the iterator (driver-supplied from `rs->V_w`).
3. `LinearSlipWeakeningIterator` `step_fn` lifts `Tpv205SubStepIterator::StepOneQP_` verbatim.
4. **Shared-rank invariant (§5.4-2)**: `RunSubSteps_` operates on the **interior** fault-QP set only; shared fault QPs keep bypassing the iterator (inline `EvaluateADER`, R-1601).

Acceptance Criteria

- `seas_test_friction_substep_iterator_parity`: on a fixed multi-QP, $O \in \{1,2,3\}$ fixture, each unified iterator reproduces its standalone counterpart bit-for-bit (`D0FData + I_imp_±`).
- After re-pointing the factory, the **SAFS+RS smoke and the LSW smoke are bit-identical** to their Phase-3 results.

Dependencies

Depends on: Phase 3. Required by: Phase 8 (TPV104 needs the SRW law added here).

9.6 Phase 6 – TPV/material/flux config schema + D3.2 fault-local stress

Goal

The TPV TOML vocabulary parses into `SpatialFrictionConfig`: the SRW state-evolution selector, `boxcar_taper` rules, `interior_flux/material.kind`, the TPV nucleation kinds, and the fault-local stress source (D3.2). (The aging RS block, the Gaussian nucleation, `constant_tensor`, and `ResolveRateState` already exist from Phases 1–3 / `safs` — this phase adds only the TPV/SRW/

flux/material extensions.)

Files to Modify

- spatial/code/spatial_friction.{hpp,cpp}, spatial/code/spatial_stress.{hpp,cpp},
dynamic/spatial_setup.hpp

Detailed Requirements

1. New blocks: ProblemSpec{tag}, BoundarySpec{fault_attr,natural_attrs,absorbing_attrs}, FaultGeometrySpec{ref_normal=(0,-1,0),up=(0,0,1),kind}, HypocenterSpec{x,y,z,nucleation_radius}, MaterialKind{Constant,DepthProfile1D,SidecarHDF5}+MaterialSpec.
2. **D3.2 fault-local stress:** extend StressSourceKind (currently {ConstantTensor, SidecarHDF5}, spatial_friction.hpp:153) with FaultLocalPrestress. Add FaultLocalPrestressSource (spatial_stress.{hpp,cpp}) with constant background {tau_strike_pa, tau_dip_pa, sigma_n_pa} (right-lateral / compression POSITIVE) + optional rectangular-tau_strike patches (center_{x,y,z}_m NaN, half_{x,y,z}_m +inf, last-match-wins). Add FaultGeometry::ComputeParams(tau_dip, sigma_n, P_p) assigning tau1_0=tau_dip, tau2_0=tau_strike, sigma_n0=sigma_n-P_p per DOF directly (no projection). (*D3.1 constant_tensor already lands in Phase 3.*)
3. [numerics]: parse cfl_safety∈{raw,dg} (default raw), fault_iterator∈{one-shot, substep} (default one-shot), interior_flux∈{scalar,matrix} (default scalar). **Mutual-exclusion guard:** mixed_flux only with interior_flux=="scalar"; matrix + mixed_flux aborts (R-1203 sibling); matrix requires material.kind!=Constant.
4. **SRW state evolution:** add StateEvolutionKind{AgingLaw,SlipLawStrongRateWeakening} (the selector does **not** exist today — aging is implicit); add RateStateBlock.{state_evolution,f_w_default, (default AgingLaw, so existing RS configs are unaffected), per-rule V_w, RateStatePerDOFParams.V_w; thread V_w through ResolveRateState (p.V_w filled per-DOF, like p.a).
5. boxcar_taper rule: extend SpatialRule::Kind (currently {Depth,Box,RegionAttribute, Barrier}, spatial_friction.hpp:177) with BoxcarTaper; add boxcar_center/half/trans_{x,y,z}_m, cohesion-taper fields, per-rule V_w; add BoxcarTaperFactor(...) and free fn SCECBoxcar(offset,half,trans). **R-114 guard:** per-rule mu_s>1e5 aborts (use kind="barrier"). SpatialRule exposes named members (r.a, r.b, r.V_w, ...) — no string field.
6. Nucleation config: extend NucleationKind (currently {GradualOverstress}, spatial_friction.hpp:219) with GradualOverstressCompactCircular (TPV102/104) and InstantaneousOverstressCircular (TPV31: center_{x,y,z}_m, radius_m, taper_m, delta_tau_pa); [nucleation] dispatches on the enabled [nucleation.<kind>] sub-block; absent [nucleation] (TPV205) → static.
7. Guards: [meta] schema_version==1 (exists); R-008 hypocenter-z (up[2]>0 ⇒ z≤0); string time forms ("0s","12s","auto"); [boundary] disjoint-set + fault_attr>0; [fault_geometry] unit-norm + non-parallel + R-003.

Acceptance Criteria

- ☐ seas_test_spatial_friction_config_phaser + seas_test_spatial_stress_with_patches pass.
- ☐ All three TPV TOMLs parse with no abort; all existing SAFS LSW/RS TOMLs still parse (the new fields default to the current behaviour — no regression).

Dependencies

Depends on: Phase 3. Required by: Phases 7, 8, 9, 10.

9.7 Phase 7 – nucleation_methods module

Goal

MakeNucleation(cfg,...) returns the correct method behind INucleationMethod; compact-circular (TPV102/104) and instantaneous-circular (TPV31) exist and match native math. (The driver keeps calling the Gaussian path directly for SAFS+RS until it adopts the factory here.)

Files to Create

- dynamic/nucleation_method.hpp, dynamic/nucleation_factory.{hpp,cpp}, tests/unit/test_nucleation

Files to Modify

- dynamic/spatial_nucleation.{hpp,cpp} (additive), drivers/spatial_dyn_driver.cpp (swap the inline ResolveGradualOverstress/nuc_cb for MakeNucleation).

Detailed Requirements

1. Add GradualOverstressCompactCircularSpec + ResolveGradualOverstressCompactCircular + ApplyGradualOverstressCompactCircularIncrement ($SCEC F(r)=\exp(r^2/(r^2-R^2))\cdot smoothStep$) — additive; leave the existing Gaussian ResolveGradualOverstress/ApplyGradualOverstress Increment (spatial_nucleation.{hpp,cpp}) untouched.
2. Add InstantaneousOverstressCircularSpec + ResolveInstantaneousOverstressCircular (one-shot cosine-tapered patch — TPV31): $\Delta\tau(r)=\Delta\tau_0$ for $r\leq R$, cosine-tapered to 0 over $R<r\leq R+taper$, 0 beyond; seeds tau{1,2}_nuc once at t=0. Math from hrs-ref TPV31.
3. Define INucleationMethod (§5.2) + concretes GaussianGradualOverstress, CompactCircularGradualOverstress, InstantaneousOverstressCircular (ApplyOnce seeds the patch; ApplyIncrement no-ops; IsPerSubStep()==false), StaticOverstress (TPV205 no-op). Factory maps cfg.nucleation.kind → concrete; absent [nucleation] → StaticOverstress.
4. Driver: replace the inline ResolveGradualOverstress (:1162) + nuc_cb lambda (:1753) with

```
auto nuc = MakeNucleation(cfg, dof_coords_3d, dof_basis); and nuc_cb=[&](real_t t,real_t dt){ nuc->ApplyIncrement(dof_data,t,dt);}; nuc->ApplyOnce(dof_data)
```

 after init for the one-shot kinds. The SAFS+RS Gaussian path is then routed through GaussianGradualOverstress (byte-identical to the inline call).

Acceptance Criteria

- ☐ seas_test_spatial_nucleation: compact-bell at $r=0(=1)$, $r=R(=0)$, mid; instantaneous at $r=0(=\Delta\tau_0)$, $r=R(=\Delta\tau_0)$, $r=R+taper(=0)$.
- ☐ seas_test_nucleation_factory: each kind (and the absent-block case) returns the expected method.
- ☐ SAFS+RS smoke is **bit-identical** to its Phase-3 result after the Gaussian path moves behind the factory.

Dependencies

Depends on: Phase 6. Required by: Phase 8, 10.

9.8 Phase 8 – TPV205/102/104 benchmarks (configs, meshes, jobs, regression)

Goal

The three primary TPV cases run through the driver (scalar Riemann) and match their SCEC references within tolerance; the review/sign/basis tests are green; $np > 1$ parity holds.

Files to Create

- `tpv{205,104,102}/configs/*.toml`, `meshes`, `jobs/tpv*_spatial/*.sbatch`; `tests/unit/test_tpv_toml_stress_sign.cpp`, `test_spatial_dyn_tpv205_review.cpp`, `test_spatial_dyn_tpv102_tpv104_review.cpp`, and the planar-TPV basis-correctness test (D1).

Detailed Requirements

1. Author the three TOMLs: (a) `[numerics] interior_flux="scalar", cfl_safety="dg"`, TPV205 `fault_iterator="one-shot"`; (b) **D3.2 fault-local stress** `kind="fault_local_prestress"` with `tau_strike_pa=+tau_ini` (TPV205 +70e6, TPV102 +75e6, TPV104 +40e6), `tau_dip_pa=0`, `sigma_n_pa=+120e6`; TPV205 adds positive `tau_strike` patches (central +81.6e6, left +78e6, right +62e6) with a header comment; (c) TPV102 `[friction.rate_state] aging + boxcar_taper`; TPV104 `state_evolution="slip_law_strong_rate_weakening" + V_w + boxcar`; both `[nucleation] gradual_overstress_compact_circular`; TPV205 `[friction.slip_weakening]`
 - barrier rules, no `[nucleation]`.
2. `test_tpv_toml_stress_sign.cpp` (D3.2): build each `FaultLocalPrestressSource`, seed a canonical-frame DOF via `ComputeParamsFaultLocal`, assert $\tau_2(\text{strike}) == \tau_{\text{ini}}$, $\tau_1(\text{dip}) == 0$, $\sigma_n == (120\text{e6} - P_p)$; precondition `dip==(0,0,-1)`, `strike==(+1,0,0)`.
3. Planar-TPV basis test (D1): build the driver-local per-QP basis on a small $y=0$ TPV mesh, assert canonical frame at every fault QP
4. Meshes (`conda activate pythonenv, v2.2`): `tpv102_1000m.msh` present; generate `gmsht-format msh22 -3 tpv104/mesh/tpv104_1000m.geo ... and ... tpv205/mesh/tpv2053d_200m.geo ...`
5. Regression: TPV station traces vs `tpv*/gold/results_*/results/*.dat` (native-driver runs) and SCEC `benchmark_data/` overlays; document tolerance; store a first `tpv*/gold_spatial/`.
6. **$np > 1$ MPI parity (§5.4-7)**: forced-2-rank single-shared-fault-face cross-rank DOFData symmetry + flux bit-identity, for LSW and rate-state.

Acceptance Criteria

- ☐ Each config `--dry-run` parses; a `tfinal=0.2s` 8-rank smoke writes SCEC `.dat` + a fault collection.
- ☐ `seas_test_tpv_toml_stress_sign`, `_spatial_dyn_tpv205_review`, `_spatial_dyn_tpv102_tpv104_review`, the planar-basis test pass; $np=2$ parity passes.
- ☐ TPV station traces match gold/SCEC within tolerance (rupture front, peak V , final slip).

Dependencies

Depends on: Phases 5, 6, 7. Required by: nothing (TPV31 is independent via Phases 9–10).

9.9 Phase 9 – Heterogeneous (matrix) Riemann solver option (PORT from hrs-ref)

Goal

interior_flux = "matrix" routes interior non-fault faces through the bimaterial Riemann flux; interior_flux = "scalar" is byte-unchanged; the fault flux is unchanged on both paths; np>1 parity holds.

Files to Port / Modify

- **Port (verbatim from hrs-ref):** dynamic/godunov_flux_bimaterial.{hpp,cpp}.
- **Modify:** dynamic/wave_operator.{hpp,inl} (add the heterogeneous ctor, the per-face pre-compute, the cross-rank material exchange, and the dispatch branches), drivers/spatial_dyn_driver.cpp (the scalar/matrixWaveOperator ctor branch + MaterialField from [material]), miniapps/seas/Makefile.

Detailed Requirements

1. Port godunov_flux_bimaterial.{hpp,cpp} exactly (the BimaterialFlux class: BuildGodunovStateFaceLocal 9×9 face-local projectors, BuildPerFaceFluxMatricesGlobal global rotation, ApplyPerFaceFlux runtime per-QP apply $F_{h_self} = fluxLocal \cdot Q_{self} + fluxNeighbor \cdot Q_{nbr}$). Keep the **SeisSol positive-stress matrix convention** (cols 0–2 positive) — required for the exact linearised bimaterial Riemann solver; the MFEM-negative convention reduces to average-flux and is wrong here.
2. Add the heterogeneous ctor WaveOperator(mesh, order, const MaterialField&, bc) that sets material_ = &material, builds owned_flux_pool_ per element via material.EvalAt(elem, T, centroid), populates per_elem_lmr_ and the per_elem_h_/MaxCpInElement CFL cache, and calls ExchangeBiMaterialNeighbours().
3. Port BuildPerFaceBimaterialFluxMatrices_() → fills per_face_bimaterial_flux_[face][side] [{fluxLocal, fluxNeighbor}] from per_elem_lmr_ and (shared faces) shared_face_neighbour_material_.
4. Wire the dispatch: in wave_operator.inl Mult (~:1311) and ADER (~:5191) interior non-fault face paths, if (owned_flux_pool_) → BimaterialFlux::ApplyPerFaceFlux(...) for each element side, else → flux_.Interior(...) (the existing scalar/mixed path). ADER CK recursion uses ApplyJacobianPerElementDOF_ when owned_flux_pool_ is set, else ApplyJacobianPerDOF.
5. **Shared-rank (§5.4):** ExchangeBiMaterialNeighbours_ is one more collective and must be gated on pmesh.GetNSharedFaces() > 0 (NOT shared-fault faces), so a rank with shared non-fault faces but no shared fault faces still participates (R-1600). **Fault faces stay on Fault-FaceFlux regardless of interior_flux** — the bimaterial path touches only interior non-fault faces; the friction iterator and ComputeADERSHaredFaceFluxRHS are unchanged.
6. **Driver scalar/matrix branch:** convert the operator to std::unique_ptr<WaveOperator> wave_ptr; and select scalar → WaveOperator(pmesh, order, λ, μ, ρ, bc) (the existing path) vs matrix → WaveOperator(pmesh, order, material_field, bc), with material_field (MaterialField) built from [material] (Constant vs DepthProfile1D, Phase 6 / Phase 10). This is where the matrix path becomes reachable; before this phase the driver only ever built the scalar ctor, so the SAFS+RS / TPV2xx scalar runs are untouched.
7. Makefile: define GODUNOV_FLUX_BIMATERIAL_{SRC,OBJ}, add to the driver link line.

Edge Cases

- `owned_flux_pool_ == nullptr` (scalar ctor) must reproduce today's scalar path **byte-for-byte** (no new branches taken) — the SAFS+RS and TPV2xx cases never select `matrix`.
- A near-equal-material face (λ, μ, ρ continuous) must reduce the bimaterial flux to the scalar Godunov flux to round-off (sanity check on the projector).
- `matrix + mixed_flux`, or `matrix + Constant` material $\rightarrow$ config-parse abort (Phase 6 mutual-exclusion guard).

Acceptance Criteria

- ☐ All scalar TPV/SAFS runs are byte-unchanged vs a pre-Phase-9 checkpoint (`owned_flux_pool_` null path untouched).
- ☐ A 2-block bimaterial unit test (`seas_test_godunov_flux_bimaterial`): `ApplyPerFaceFlux` reproduces the hrs-ref reference values for a known material jump.
- ☐ $np > 1$ parity on a heterogeneous mesh: interior-face flux is bit-identical across the rank seam (`cross-rank ExchangeBiMaterialNeighbours_ correct`).

Dependencies

Depends on: Phases 3, 6. Required by: Phase 10.

9.10 Phase 10 – TPV31 (depth-heterogeneous LSW) verification case (PORT from hrs-ref)

Goal

TPV31 runs through `spatial_dyn_driver` on the `matrix` Riemann path with a 1-D depth-profile material and matches the SCEC eqdyna/seisol references.

Files to Port / Create

- **Port:** `DepthProfile1D/MakeDepthProfile1DMaterial` into `dynamic/heterogeneous_material.{hpp,cpp}`.
- **Copy (from hrs-ref):** the `tpv31/` tree — `tpv31/configs/tpv31.toml`, `tpv31/mesh/tpv31_50m.{geo,msh}`, `tpv31/benchmark_document/`, `tpv31/benchmark_data/`, `tests/unit/test_tpv31_*.cpp`.
- **Create:** `jobs/tpv31_spatial/tpv31_spatial_dyn_50m_*.sbatch`.

Detailed Requirements

1. Port `MakeDepthProfile1DMaterial(layers, depth_axis)` verbatim (Coefficient-mode `MaterialField`; `depth = max(0, -z)` along the depth axis; discontinuous layers with linear interpolation between specified depths). Wire `MaterialKind::DepthProfile1D` (parsed in Phase 6) $\rightarrow$ this builder $\rightarrow$ the `matrix WaveOperator` ctor (Phase 9).
2. Adapt `tpv31.toml` to the spatial schema: `[meta].law = "lsw"` (`fault_geometry.kind = tpv31_lsw`), `[material] kind = "depth_profile_1d" + [[material_profile.layer]]`, `[numerics] interior_flux = "matrix", cfl_safety = "dg"`; `[friction.slip_weakening]` with $\mu_s=0.58$, $\mu_d=0.45$, $d_c=0.18$; **depth-linear cohesion** $C_0(\text{depth}) = 0.000425 \text{ MPa/m} \cdot \max(0, 2400 - \text{depth})$; `[nucleation] instantaneous_overstress_circular` (hypocenter $z=-7500 \text{ m}$, `radius/taper/ $\Delta\tau_0$` from the SCEC spec).
3. Wire per-DOF cohesion $C_0(\text{depth})$ into the LSW resolver (`ResolveSlipWeakening`) and into the LSW friction kernel's strength $\mu_s \cdot \sigma_n + C_0$ (additive; if safs's LSW path lacks a cohesion term, add it guarded so $C_0=0$ reproduces the current behaviour exactly).

4. Mesh (conda activate pythonenv): use the committed v2.2 tpv31_50m.msh, or regenerate `gmsh -format msh22 -3 tpv31/mesh/tpv31_50m.geo -o tpv31/mesh/tpv31_50m.msh`.
5. Verification: TPV31 station traces vs `tpv31/benchmark_data/scec_{eqdyna,seisol}`; document tolerance and store a `tpv31/gold_spatial/` set from the validated run.

Edge Cases

- Depth-axis sign: $up[2]>0 \Rightarrow z \leq 0$ (R-008); the profile uses $depth = -z$.
- $C0=0$ must reproduce the pre-cohesion LSW path byte-for-byte (the cohesion term is additive and zero for the other TPV cases).

Acceptance Criteria

- ☐ `tpv31.toml --dry-run --verify-dispatch` reports matrix Riemann + `depth_profile_1d` material + `tpv31_lsw` + `instantaneous overstress_circular`.
- ☐ An 8-rank smoke completes and writes SCEC station output; full run station traces match the `eqdyna/seisol` overlays within the documented tolerance.
- ☐ `seas_test_tpv31_*` (canonical rotation, nucleation+cohesion, station writer) pass.

Dependencies

Depends on: Phases 6, 7, 9.

10 Testing strategy

- » **Near-future milestone (Phases 1–3):** `tpv102_nuc_callback_parity` (byte-safe overload, **R-008 method-pinned** + a Brent smoke) + `seed_equilibrium_psi_rs` (steady-residual, **R-001 d.eta_s + v_init>0**, **R-009 V0-guard** case) + `advance_interface_compiles` (**R-010 compile-only**) + `friction_iterator_factory` (dispatch) + `rate_state_config_no_abort` (**R-002 dry-run**) + `resolve_rate_state_guards` (**R-003 no-double-PP**, **R-006 per-DOF reject**, **R-011 a>b allowed**) + `paraview_state_channel_rs` (**R-005**) + `constant_tensor_sign` (**R-007** parametrized over all flipped configs) gate the SAFS+RS wiring; the deliverable is the **8-rank SAFS+RS smoke** (locked at `V_init` pre-nucleation, physical rupture after), with the **R-004 two-pronged** ψ -consistency check (cross-rank bit-identity + temporal interior-vs-shared drift) as the open risk gate.
- **Config + sign contract (cheap, deterministic, no MPI):** `tpv_toml_stress_sign` (D3.2), `constant_tensor_sign` (D3.1 bit-identity, parametrized), `resolve_rate_state_guards`, `*_review`, `config_phaser`, `stress_with_patches`, the factory tests — these gate every phase and are the practical regression without an HPC reference.
- **Unification parity (the byte-exactness gate):** `friction_substep_iterator_parity` asserts the unified iterators reproduce the standalone `tpv{205,102,104}_substep_iterator` (the oracle, kept untouched) bit-for-bit across $O \in \{1,2,3\}$.
- **Riemann unit test:** `godunov_flux_bimaterial` checks `ApplyPerFaceFlux` against hrs-ref reference values and against the scalar Godunov flux in the equal-material limit.
- **Dispatch smoke:** `--dry-run/--verify-dispatch` confirm law + iterator + nucleation + flux + material + station writer per case (incl. TPV31 $\rightarrow$ matrix + `depth_profile_1d`).

- **Physics regression:** 8-rank $t_{\text{final}}=0.2\text{s}$ smoke then full run (cluster) vs gold/SCEC for TPV205/102/104; TPV31 vs the eqdyna/seisol overlays. The standalone `tpv*_driver.cpp` remain available for same-mesh cross-checks (the byte-exactness oracle).
- **MPI parity ($np>1$):** forced-2-rank single-shared-fault-face cross-rank D0FData/flux bit-identity (friction); heterogeneous-mesh interior-face flux bit-identity (bimaterial).
- **SAFS guards:** LSW no-regression diff (incl. the D3.1 config-sign flip $\rightarrow$ byte-identical); RS shared-fault ψ -consistency — **R-004 two-pronged:** `shared_fault_rs_psi_consistency_np2` checks (a) cross-rank `d.psi` bit-identity post-reconcile **and** (b) bounded temporal interior-vs-shared ψ drift at the same QP.

11 Risk assessment

Risk	Likelihood	Detection / mitigation
Shared-fault RS ψ 1st-order at $O \geq 2$ (review R-004)	MED-HIGH	Bounded (mirrors the working LSW slip path; reconcile broadcasts ψ so it is <i>temporal</i> , not desync). Phase-3 two-pronged gate: cross-rank bit-identity + temporal interior-vs-shared drift; $np>1$ RS stays “open until validated”; may open a sub-step- ψ -on-shared-QPs follow-up; do not assume LSW correctness transfers
RS code won’t compile / aborts (review R-001/R-002/R-010)	(closed)	R-001 <code>d.eta_s</code> (not <code>rs.eta_s</code>) + include + <code>V_init>0</code> guard; R-002 gate <code>slip_weakening.has_value()</code> under <code>is_lsw</code> , outer-scope <code>lsw+rs</code> ; R-010 rename <code>:439</code> $\rightarrow$ Advance + add <code>GetDeltaT/GetTimeWeights</code> to the interface. Caught by <code>seed_equilibrium_psi_rs</code> (build) + <code>advance_interface_compiles</code> + <code>rate_state_config_no_abort</code> (dry-run)

Risk	Likelihood	Detection / mitigation
Dual-source V0: silent wrong friction if $V_0 \neq 1e-6$ (review R-009)	MED (CRIT if tuned)	Force solve hardcodes <code>FrictionSolver::V0;</code> seed+aging use config <code>V_0</code> . Phase-1 guard asserts equality (deliverable uses $1e-6$); proper fix = thread config V0 into the solver (gated follow-up, no-hardcoded-numbers). Caught by <code>seed_equilibrium_psi_rs</code> V0 case
Latent RS inconsistencies (review R-003/R-005/R-006)	(closed)	R-003 <code>PorePressureSpec{}</code> (no double-PP); R-005 ParaView state $\rightarrow \psi$; R-006 reject per-DOF <code>b/V_0/f_0</code> . Caught by <code>resolve_rate_state_guards + paraview_state_channel_rs</code>
a<b validator blocks a bounded rupture (review R-011)	LOW	Fully-VW first smoke is unaffected; Phase-3 relaxes <code>a<b</code> $\rightarrow$ allow <code>a>b</code> (keep <code>a, b > 0</code>) so a <code>tpv102</code> -like arrest border is configurable. Caught by <code>resolve_rate_state_allows_velocity_stren</code>
Matrix Riemann: scalar path regresses during the port	MED	scalar runs byte-unchanged vs pre-Phase-9 checkpoint; <code>owned_flux_pool == nullptr</code> takes no new branch
Matrix Riemann: SeisSol vs MFEM matrix sign convention	MED	equal-material limit must reduce to scalar Godunov to round-off; <code>godunov_flux_bimaterial</code> unit test vs hrs-ref values
New bimaterial collective deadlocks (R-1600)	MED	gate <code>ExchangeBiMaterialNeighbours_</code> on <code>GetNSharedFaces() > 0</code> , not shared-fault faces; <code>np > 1</code> parity test
Driver integration drops a safs fix	MED	§5.5 “stays” checklist + SAFS LSW no-regression diff

Risk	Likelihood	Detection / mitigation
Unified iterator drifts from the oracle	LOW	friction_substep_iterator_parity bit-for-bit vs standalone iterators; standalone iterators untouched
D3.1 half-applied (factory negation XOR config flip)	LOW	constant_tensor_sign golden test fails loudly on a half-done change; land both together
D3.2 fault-local seeding wrong (slot / sign)	LOW	direct seeding (no projection); tpv_toml_stress_sign asserts tau2_0==+tau_ini + canonical frame
TPV31 cohesion term perturbs other cases	LOW	C0=0 reproduces the pre-cohesion LSW path byte-for-byte (additive, guarded)
Equilibrium- ψ seed wrong (creep/lock at t=0)	LOW	seed_equilibrium_psi_rs residual test (Phase 1) + the SAFS-RS smoke lock check (Phase 3)
Missing .msh for tpv104/tpv205/tpv31	HIGH (known)	Phase-8/10 gmsh step is a hard prerequisite (does NOT block SAFS+RS, which uses the existing 500 m mesh)
Reference identifier drift (DieterichRuinaFriction::Constants order; Tpv102 overload insertion; hrs-ref BimaterialFlux API)	LOW	confirm at impl time; flagged in Phases 1/3/9/10

12 Appendix

- **PDF build:** ./document/parts_dev/_assets/md2pdf.sh document/fullelasticity_dev/PLAN_tpv_regress (Charter/Avenir/Menlo, TOC, numbered sections; Unicode math via pdf-style.tex).
- **Reference clone:** /Users/chunhuizhao/projects/seas-mfem-heterogenous-riemann-solver (feature/heterogeneous_riemann_solver, e30d003); fetched here as tag hrs-ref; merge-base 0e13b25.
- **Files ported verbatim from hrs-ref (Phases 9–10):** dynamic/godunov_flux_bimaterial.{hpp, cpp} (BimaterialFlux), the owned_flux_pool_dispatch in wave_operator.inl (Mult ~:1311 / ADER ~:5191) + BuildPerFaceBimaterialFluxMatrices_ + ExchangeBiMaterialNeighbours_, MakeDepthProfile1DMaterial in heterogeneous_material.*, and the tpv31/ tree (config/mesh/benchmark/tests). Reproduce diffs with git diff 0e13b25 hrs-ref -- miniapps/seas/dynamic/godunov_flux_bimaterial.hpp etc.

- **Reference RS-for-SAFS plan:** safs/project_7.0_alternative/document/PLAN_rate_state_friction_opti
(used as a reference; identifier corrections noted in Phases 1/3/6).
- **Analysis archives:** /tmp/branchcmp/agent{1..5}*.md, /tmp/branchcmp/pass2*.md.